Python: Recap IV

This notebook was generated in **student** mode.

Exercise 1: ASCII sum substring

Exercise Description

You are given a number `N` and a string `S`. In Python, a character can be converted to a numeric code (its ASCII representation) using the function `ord()`. For example, `ord('A')` is 65, `ord('B')` is 66, `ord('a')` is 97, `ord('1')` is 49, and so on.

Write a function `ascii_sum(N, S)` that returns `True` if `S` contains a substring whose **sum of ASCII codes** is exactly `N`, and `False` otherwise.

Example: if `N = 180` and `S = 'CAB19'`, the function must return `True` because there exists the substring `'AB1'` such that `ord('A') + ord('B') + ord('1') = 65 + 66 + 49 = 180`.

Your solution

```python
def ascii_sum(N, S):
    # Write your solution here
    pass
```

Test your solution

```python
# Test case 1 (Example)
N = 180
S = 'CAB19'
assert ascii_sum(N, S) is True

# Test case 2
N = 180
S = 'CAB91'
assert ascii_sum(N, S) is False

# Test case 3
N = 200
S = 'Hello, World!'
assert ascii_sum(N, S) is False

# Test case 4
N = 65  # ASCII code for 'A'
S = 'ABCDEF'
assert ascii_sum(N, S) is True
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

Exercise 2: Flip if upper

Exercise Description

Write a function `my_function(s)` that receives a string `s` (a sentence where words are separated by whitespaces) and returns a new string built according to the following rule: if a word in `s` begins with an uppercase letter, then the word must be **reversed** in the new string; otherwise it must be left intact.

The order of the words between the input and the output string must not change. If the input string is empty, the function must return `None`.

Note: the examples include a trailing space at the end of the returned string.

Your solution

```
def my_function(s):
    # Write your solution here
    pass
```

Test your solution

```
# Test case 1 (Example)
s = 'hello World'
t = my_function(s)
assert t == 'hello dlroW '

# Test case 2
s = ''
t = my_function(s)
assert t is None

# Test case 3
s = 'Python is AWESOME'
t = my_function(s)
assert t == 'nohtyP is EMOSEWA '

# Test case 4
s = 'viva david parenzo'
t = my_function(s)
assert t == 'viva david parenzo '
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

Exercise 3: Longest word in string

Exercise Description

Write a function `my_function(s)` that receives a string `s` (representing a sentence where words are separated by whitespaces) and returns the **longest word** in `s`.

If the input string is empty, the function must return `None`. If there are more than one longest words, the function must return the **first** one.

Your solution

```
def my_function(s):
    # Write your solution here
    pass
```

Test your solution

```
# Test case 1 (Example)
s = 'This is an example sentence with some long words.'
assert my_function(s) == 'sentence'

# Test case 2
s = 'The quick brown fox jumps over the lazy dog.'
assert my_function(s) == 'quick'
```

```
# Test case 3
s = ''
assert my_function(s) is None

# Test case 4
s = 'Lorem ipsum dolor sit amet, consectetur adipiscing elit. Nunc tincidunt vitae mi
assert my_function(s) == 'consectetur'
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

Exercise 4: Periodic String Group

Exercise Description

Write a function `my_function(X, Y, S)` that, given two integer values `X` and `Y` and a string `S` of length 1 (a character), returns a string with `Y` **groups** of characters. Each group consists of `X` copies of `S`. Groups are separated by `'-'`.

You can assume that `X` and `Y` are greater than or equal to 0. If `X == 0`, groups are empty and no `'-'` separators should be used (the function should return the empty string).

Hint: you can use the `*` operator to obtain sequences of equal characters, e.g., `'x' * 7 == 'xxxxxxx'`.

Your solution

```python
def my_function(X, Y, S):
    # Write your solution here
    pass
```

Test your solution

```python
# Test case 1 (Example)
s1 = my_function(4, 2, 'A')
assert s1 == 'AAAA-AAAA'

# Test case 2
s1 = my_function(1, 10, 'A')
assert s1 == 'A-A-A-A-A-A-A-A-A-A'

# Test case 3
s1 = my_function(0, 5, 'A')
assert s1 == ''

# Test case 4
s1 = my_function(3, 5, 'A')
assert s1 == 'AAA-AAA-AAA-AAA-AAA'
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

Exercise 5: Periodic String

Exercise Description

Write a function `my_function(X, Y, S)` that, given two integer values `X` and `Y` and a string `S` of length 1 (a character), returns a string with `X` characters `S` separated by `Y` characters `'-'`. You can assume that `X` and `Y` are greater than or equal to `0`.

If `X == 1` no `'-'` characters should be added, independently of the value of `Y`.

Hint: you can use the `*` operator to obtain sequences of equal characters (e.g., `'x' * 7 == 'xxxxxxx'`).

Your solution

```
def my_function(X, Y, S):
    # Write your solution here
    pass
```

Test your solution

```
# Test case 1 (Example)
s1 = my_function(3, 2, 'A')
assert s1 == 'A--A--A'

# Test case 2
s1 = my_function(1, 10, 'A')
assert s1 == 'A'

# Test case 3
s1 = my_function(0, 5, 'A')
assert s1 == ''

# Test case 4
s1 = my_function(4, 5, 'A')
assert s1 == 'A-----A-----A-----A'
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

Exercise 6: Remove Char

Exercise Description

Write a function `my_function(S, C)` that, given as parameters a string `S` and another string `C` of length `1` (a character), returns a string `T` that is equal to `S` except that it contains **no occurrence** of character `C`.

Your solution

```python
def my_function(S, C):
    # Write your solution here
    pass
```

Test your solution

```python
# Test case 1 (Example)
s = 'I study at Luiss'
c = 's'
t = my_function(s, c)
assert t == 'I tudy at Lui'

# Test case 2
s = 'xxxxxxo'
c = 'x'
t = my_function(s, c)
assert t == 'o'
```

```
# Test case 3
s = 'oxxxxxxx 000'
c = 'x'
t = my_function(s, c)
assert t == 'o 000'

# Test case 4
s = 'oxxxxxxxo'
c = 's'
t = my_function(s, c)
assert t == 'oxxxxxxxo'
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

Exercise 7: Replace Substring

Exercise Description

Write a function `my_function(S, R, C)` that, given as parameters a string `S`, a string `R` and another string `C` of length `1` (a character), returns a string `T` that is equal to `S` except that each occurrence of character `C` is **replaced** by string `R`.

Your solution

```python
def my_function(S, R, C):
    # Write your solution here
    pass
```

Test your solution

```python
# Test case 1 (Example)
s = 'I study at Luiss'
r = 'REPLACE'
c = 's'
t = my_function(s, r, c)
assert t == 'I REPLACEtudy at LuiREPLACEREPLACE'

# Test case 2
s = 'I REPLACEddd'
r = 'REPLACE'
c = 'D'
t = my_function(s, r, c)
assert t == 'I REPLACEddd'

# Test case 3
s = 'D'
r = 'REPLACE'
c = 'D'
t = my_function(s, r, c)
assert t == 'REPLACE'
```

```python
# Test case 4
s = ''
r = 'REPLACE'
c = 'D'
t = my_function(s, r, c)
assert t == ''
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```python
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

Exercise 8: Driving License

Exercise Description

Write a function `my_function(license_plate)` that validates a driving license plate format.

The function should return `True` if the license plate follows the correct format, and `False` otherwise.

A valid license plate must:

- Be exactly 7 characters long
- Have exactly 4 letters followed by exactly 3 digits
- All letters must be uppercase

For example: "ABCD123" is valid, but "ABC123", "ABCDE123", or "ABCD12" are not valid.

Your solution

```python
# Write your solution here
```

Test your solution

```python
# Test case 1 (Example)
result = my_function("ABCD123")
assert result == True

# Test case 2
result = my_function("ABC123")
assert result == False

# Test case 3
result = my_function("ABCDE123")
assert result == False

# Test case 4
result = my_function("ABCD12")
assert result == False
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

Exercise 9: Reverse Order

Exercise Description

Write a Python function `my_function(s)` that takes a string `s` as an argument and returns a new **list of words** that contains the words in `s` in reverse order.

For example, if `s = 'I love you'`, then `my_function(s)` should return `['you', 'love', 'I']`.

Hint: you can use the `split()` method to split the string into a list of words.

Your solution

```
def my_function(s):
    # Write your solution here
    pass
```

Test your solution

```python
# Test case 1 (Example)
s1 = 'I love you'
assert my_function(s1) == ['you', 'love', 'I']

# Test case 2
s2 = 'The quick brown fox'
assert my_function(s2) == ['fox', 'brown', 'quick', 'The']

# Test case 3
s3 = 'Hello World'
assert my_function(s3) == ['World', 'Hello']

# Test case 4
s4 = 'Python is fun'
assert my_function(s4) == ['fun', 'is', 'Python']
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```python
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

Exercise 10: Alien race

Exercise Description

It's 2027. Earth has been invaded by three alien races: the Greys, the Reptilians, and the Annunaki. A running race is organized to decide which race will dominate. At the end of the race, a list is provided with the order in which the participants finished. Each element of the list is in the format `participant-race`.

The score of a race is determined by **adding the positions** of all its participants, and the race with the **lowest score** wins.

Write a function `my_function(order_of_arrival, race)` that calculates the score obtained by a specific race. Positions are 1-based (the first element has position 1). If the list does not include that race among the participants, the result should be `0`.

Your solution

```
def my_function(order_of_arrival, race):
    # Write your solution here
    pass
```

Test your solution

```
# Test case 1
order_of_arrival = ['Gix-Annunaki', 'Tom-Grey', 'XYZ-Reptilian', 'Alex-Annunaki', 'Ar
race = 'Annunaki'
assert my_function(order_of_arrival, race) == 22

# Test case 2
```

```
race = 'Grey'
assert my_function(order_of_arrival, race) == 26

# Test case 3
order_of_arrival = ['Gix-Annunaki', 'Tom-Grey', 'Alex-Annunaki', 'AngelFab-Grey', 'Mi
race = 'Reptilian'
assert my_function(order_of_arrival, race) == 0

# Test case 4
order_of_arrival = ['Gix-Annunaki', 'Tom-Grey', 'XYZ-Reptilian', 'Alex-Annunaki', 'Ar
race = 'Humans'
assert my_function(order_of_arrival, race) == 0
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

Exercise 11: Remove numeric duplicates and sort

Exercise Description

Write a function `my_function(L)` that, given a list of numbers `L`, returns a new list containing the same numbers **without duplicates**.

The numbers in the output list must be sorted from the smallest to the largest. If the input list is empty, the function should return an empty list.

Your solution

```
def my_function(L):
    # Write your solution here
    pass
```

Test your solution

```
# Test case 1 (Example)
assert my_function([10, 2, 3, 5, 2, 10, 3]) == [2, 3, 5, 10]

# Test case 2
assert my_function([]) == []

# Test case 3
assert my_function([1, 1, 1, 1, 1, 1]) == [1]

# Test case 4
assert my_function([1, -2, 3, -4, 5, -6, 7, -8, 9, -10] * 1000) == [-10, -8, -6, -4,
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

Exercise 12: Euro To Dollars

Exercise Description

Write a function `my_function(L)` that is given a list `L` of numbers representing coins in euro. The function should return the **sum of the coins converted to dollars**, where 1 euro corresponds to 1.22 dollars.

If there are no items in the list, the function should return `0.0` (a float).

Your solution

```
def my_function(L):
    # Write your solution here
    pass
```

Test your solution

```
# Test case 1 (Example)
assert my_function([3, 2, 4]) == 10.98
```

```python
# Test case 2
assert my_function([0, 0, 1]) == 1.22

# Test case 3
assert my_function([2, 5, 12]) == 23.18

# Test case 4
assert float(my_function([])) == 0.0
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```python
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

Exercise 13: H-index

Exercise Description

The *h*-index is a common index to quantify the scientific impact of a researcher. For a given researcher, you are given a list `papers` of `n` integer numbers, where `n` is the number of papers. In position *i* you have the number of citations of the *i*-th paper.

The *h*-index is defined as the largest number `h` such that there are **at least `h` papers with at least `h` citations**.

Write a function `my_function(papers)` that returns the h-index of the researcher. You can assume the list contains only integers. If the input list is empty, the function must return `None`.

Your solution

```
def my_function(papers):
    # Write your solution here
    pass
```

Test your solution

```
# Test case 1 (Example)
assert my_function([24, 2, 36, 2, 3]) == 3

# Test case 2
assert my_function([3, 0, 6, 1, 5]) == 3

# Test case 3
assert my_function([]) is None

# Test case 4
assert my_function([24, 6, 12, 36, 2, 2]) == 4
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

Exercise 14: Km To Miles

Exercise Description

Write a function `my_function(L)` that is given a list `L` of numbers representing distances in kilometers. The function should return the **sum of the distances converted to miles**, where 1 kilometer corresponds to 0.62 miles.

If there are no items in the list, the function should return `0`.

Your solution

```
def my_function(L):
    # Write your solution here
    pass
```

Test your solution

```
# Test case 1 (Example)
assert my_function([3, 2, 4]) == 5.58
```

```
# Test case 2
assert my_function([3]) == 1.8599999999999999

# Test case 3
assert float(my_function([0])) == 0.0

# Test case 4
assert float(my_function([])) == 0.0
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

Exercise 15: Name Ends With

Exercise Description

Write a function `my_function(L1, L2)` that takes two lists as parameters:

- each element in `L1` is the name of a person,
- each element in `L2` is a character (a string of length 1).

The function shall return a list `L3` containing the persons whose name **ends with** a letter in `L2`. Elements in `L3` should appear in the same order in which they appear in `L1`.

If `L1` is empty or does not contain any person whose name ends with a letter in `L2`, then `L3` must be an empty list.

Your solution

```python
def my_function(L1, L2):
    # Write your solution here
    pass
```

Test your solution

```python
# Test case 1 (Example)
assert my_function(["Giorgio", "Irene", "Antonio", "Carla"], ["o", "e"]) == ["Giorgic

# Test case 2
assert my_function(["Giorgio", "Irene", "Antonio", "Carla"], ["e", "x"]) == ["Irene"]

# Test case 3
assert my_function(["Giorgio", "Irene", "Antonio", "Carlx"], ["e", "x"]) == ["Irene",

# Test case 4
assert my_function(["Giorgio", "Irene", "Antonio", "Carlx"], ["v", "z"]) == []
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

Exercise 16: Nevio the Ironed

Exercise Description

"Nevio the Ironed" is a fervent gambler who has accumulated a series of debts with banks. He is ordered to repay the amounts owed. Nevio currently has in his pocket a `winning_sum` of money he just won by betting on a horse.

You are given a list `debts` where each element has the format `'amount-repaid'`:

- `amount` is an integer amount of money,
- `repaid` is `0` if the amount has **not** yet been settled and `1` if it has already been repaid.

Write a function `my_function(debts, winning_sum)` that returns `True` if Nevio can settle **all unpaid debts** using `winning_sum`, and `False` otherwise.

Your solution

```python
def my_function(debts, winning_sum):
    # Write your solution here
    pass
```

Test your solution

```python
# Test case 1
debts = ['300-1', '200-0', '100-0']
winning_sum = 500
assert my_function(debts, winning_sum) is True

# Test case 2
debts = ['500-0', '200-0', '100-1']
winning_sum = 500
assert my_function(debts, winning_sum) is False

# Test case 3
debts = ['500-0', '200-0', '100-1']
winning_sum = 400
assert my_function(debts, winning_sum) is False

# Test case 4
debts = ['500-0', '200-0', '100-1']
winning_sum = 300
assert my_function(debts, winning_sum) is False
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

Exercise 17: No escape for Earth

Exercise Description

Some theories suggest the existence of a planet called Nibiru or Planet X. Scientists have identified a series of meteors approaching Earth. For each meteor, they recorded its radius and whether its trajectory is directed towards Earth.

Data are given in a list `data`, where each element has the format `'radius-trajectory'`:

- `radius` is a number,
- `trajectory` is `0` if it is **not** directed towards Earth, and `1` otherwise.

You are also given:

- `circumference`: the **maximum acceptable circumference** for a meteor,
- `limit`: the **maximum number of meteors** directed towards Earth that we can withstand.

Write a function `my_function(data, circumference, limit)` that returns:

- `False` if **any** meteor has circumference greater than `circumference`, or if the number of meteors with trajectory `1` is greater than `limit`,
- `True` otherwise.

You can use `math.pi` and the formula `circ = radius * radius * pi`.

Your solution

```
import math

def my_function(data, circumference, limit):
    # Write your solution here
    pass
```

Test your solution

```
# Test case 1
data = ['100-1', '200-1', '300-0', '400-1', '500-0']
circumference = 400
limit = 2
assert my_function(data, circumference, limit) is False

# Test case 2
limit = 1
assert my_function(data, circumference, limit) is False

# Test case 3
```

```
limit = 3
assert my_function(data, circumference, limit) is False

# Test case 4
circumference = 3000000
limit = 4
assert my_function(data, circumference, limit) is True
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

Exercise 18: Person and first letter

Exercise Description

Write a function `my_function(L1, L2)` that takes two lists as parameters:

- each element in list `L1` is the name of a person,
- each element in list `L2` is a character (a string of length `1`).

The function shall return a list L3 containing the persons whose name **begins with** a letter ir L2. Elements in L3 should appear in the same order in which they appear in L1.

If L1 is empty or does not contain persons whose name begins with a letter in L2, then L3 must be an empty list.

Your solution

```
def my_function(L1, L2):
    # Write your solution here
    pass
```

Test your solution

```
# Test case 1 (Example)
assert my_function(['Giorgio', 'Irene', 'Antonio', 'Giuseppe'], ['I', 'G']) == ['Gior

# Test case 2
assert my_function(['Giorgio', 'Mino', 'Antonio', 'Marco'], ['I', 'M']) == ['Mino', '

# Test case 3
assert my_function(['Giorgio', 'Mino', 'Antonio', 'Marco'], ['I', 'S']) == []

# Test case 4
assert my_function([], ['I', 'M']) == []
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

Exercise 19: Products of Elements

Exercise Description

Write a function `my_function(L1, L2)` that takes as parameters two lists of integer numbers.

The function shall return an empty list if `L1` and `L2` are empty **or** do not have the same length. Otherwise, it shall return a list `L3` containing the products of elements in the same position in `L1` and `L2`.

Your solution

```
def my_function(L1, L2):
    # Write your solution here
    pass
```

Test your solution

```python
# Test case 1 (Example)
assert my_function([10, 20, 30], [100, 200, 300]) == [1000, 4000, 9000]

# Test case 2
assert my_function([10, 30], [100, 300]) == [1000, 9000]

# Test case 3
assert my_function([10, 30, 15], [100, 300]) == []

# Test case 4
assert my_function([], []) == []
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```python
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

Exercise 20: Remove duplicates (strings)

Exercise Description

Define a function `my_function(lst)` that takes a list of strings `lst` as input.

The function shall return a list containing the same elements of `lst`, but **without duplicates**: if a string `s` appears more than once in `lst`, only the **first** occurrence of `s` should be copied to the output list.

If the input list `lst` is empty, the function should return an empty list.

Your solution

```python
def my_function(lst):
    # Write your solution here
    pass
```

Test your solution

```python
# Test case 1 (Example)
a = ['a', 'good', 'day', 'is', 'good', 'a', 'bad', 'day', 'is', 'not', 'good']
assert my_function(a) == ['a', 'good', 'day', 'is', 'bad', 'not']

# Test case 2
a = ['test', 'test', 'test', 'test', 'test', 'test', 'test', 'test']
assert my_function(a) == ['test']

# Test case 3
a = ['test', 'test', 'hello', 'test', 'hello', 'test', 'hello', 'test']
assert my_function(a) == ['test', 'hello']

# Test case 4
a = []
assert my_function(a) == []
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

Exercise 21: Reptilians

Exercise Description

The Reptilians are among us! An intelligence agency has revealed that many presidents of the world's nations are actually clones controlled by the Reptilians. The clones have specific characteristics:

- They have odd-length names.
- They have light-colored eyes.
- They are over 40 years old.

Each president is represented as a string in the format `'name-eyes-age'`, where `eyes` can be either `'light'` or `'dark'`.

Write a function `my_function(presidents)` that returns the number of presidents that are clones according to the rules above.

Your solution

```python
def my_function(presidents):
    # Write your solution here
    pass
```

Test your solution

```python
# Test cases 1
presidents = ['John-light-45', 'George-dark-54', 'Barack-dark-55', 'Bill-light-68', '
assert my_function(presidents) == 1

# Test cases 2
presidents = ['John-light-45', 'George-dark-54', 'Barack-dark-55', 'Bill-light-68', '
assert my_function(presidents) == 2

# Test cases 3
presidents = ['John-light-40', 'George-dark-54', 'Barack-dark-55', 'Bill-dark-68', 'F
assert my_function(presidents) == 0

# Test cases 4
presidents = []
assert my_function(presidents) == 0
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

Exercise 22: Rugby betting scandal

Exercise Description

In New Zealand, a rugby betting scandal has erupted. Some players have placed bets on certain matches despite the rules prohibiting it. The rules forbid betting from any **illegal** platform, and there is also a maximum amount of money that can be spent on legal platforms.

The federation has gathered a list `bets` of bets placed by players, where each element has the format `'name-bet-illegal'`:

- `name` is the name of the rugby player,
- `bet` is the amount wagered (a float),
- `illegal` is `0` if the site was legal, and `1` otherwise.

Write a function `my_function(bets, player)` that returns `True` if the player should be **disqualified**, and `False` otherwise.

A player is disqualified if:

- they placed **any** bet on an illegal platform, or
- the **total** amount they bet on legal platforms exceeds 500.

Your solution

```
def my_function(bets, player):
    # Write your solution here
    pass
```

Test your solution

```
# Test case 1
bets = ['Beans-30.5-0', 'Beans-20.4-1', 'Tonal-50.5-1', 'Tonal-100.5-0', 'TheShara-21
player = 'Beans'
assert my_function(bets, player) is True

# Test case 2
player = 'Tonal'
assert my_function(bets, player) is True

# Test case 3
player = 'TheShara'
assert my_function(bets, player) is False

# Test case 4
player = 'Mikasa'
assert my_function(bets, player) is False
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

Exercise 23: Subtract digit sum

Exercise Description

Write a function `subtract_digits(n)` that, given an integer number `n`, returns the number `d` obtained by **subtracting from `n` the sum of its digits**.

Hint: convert `n` into a string and work on each character, which corresponds to a digit.

Example: if `n = 12`, the function should return `9` because `12 - 1 - 2 = 9`.

Your solution

```
def subtract_digits(n):
    # Write your solution here
    pass
```

Test your solution

```python
# Test case 1 (Example)
assert subtract_digits(12) == 9

# Test case 2
assert subtract_digits(1038) == 1026

# Test case 3
assert subtract_digits(0) == 0

# Test case 4
assert subtract_digits(1199) == 1179
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```python
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

Exercise 24: Sum larger or equal 3

Exercise Description

Write a function `my_function(L)` that, given a list `L` of strings, returns the **sum of the lengths** of the strings whose length is **greater than or equal to 3**.

If there are no items in the list or no strings of length ≥ 3, the function should return `0`.

Your solution

```
def my_function(L):
    # Write your solution here
    pass
```

Test your solution

```
# Test case 1 (Example)
assert my_function(['luiss', 'row', 'a', 'bx']) == 8

# Test case 2
assert my_function(['a', 'a', 'a', 'a']) == 0

# Test case 3
assert my_function(['aa', 'aaaa', 'aaa', 'aaa']) == 10

# Test case 4
assert my_function([]) == 0
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```python
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

Exercise 25: Sum of Even Squares

Exercise Description

Write a function `my_function(L)` that, given a list `L` of numbers, returns the **sum of the squares of the even numbers** contained in `L`.

If there are no items or no even numbers in the list, the function should return `0`.

Your solution

```python
def my_function(L):
    # Write your solution here
    pass
```

Test your solution

```python
# Test case 1 (Example)
assert my_function([2, 13, 8, 5]) == 68

# Test case 2
assert my_function([1, 3, 5, 7]) == 0

# Test case 3
```

```
assert my_function([3, 101, 9, 7, 4]) == 16

# Test case 4
assert my_function([]) == 0
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

Exercise 26: Sum of all the odd

Exercise Description

You have an integer number `n` as input. Define a function `all_odd(n)` that returns the **sum of all odd integer numbers between 1 and `n`**, `n` included.

Your solution

```
def all_odd(n):
    # Write your solution here
    pass
```

Test your solution

```
# Test case 1 (Example)
assert all_odd(7) == 16

# Test case 2
assert all_odd(6) == 9

# Test case 3
assert all_odd(100) == 2500

# Test case 4
assert all_odd(1000 * 1000) == 250000000000
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

Exercise 27: Valid votes

Exercise Description

You are given a list of votes `votes` and a candidate name `candidate`. Each vote is represented as a string in the format `'candidate-valid_vote'`, where `valid_vote` is either `0` or `1` (`1` means the vote is valid).

Write a function `my_function(votes, candidate)` that **counts how many valid votes** (with `valid_vote == '1'`) the given candidate received and returns that count. The candidate may have received 0 votes.

Your solution

```
def my_function(votes, candidate):
    # Write your solution here
    pass
```

Test your solution

```
# Test case 1
votes = ['A-1', 'A-0', 'B-1', 'B-1', 'C-0', 'A-1', 'C-1', 'D-0']
assert my_function(votes, 'A') == 2

# Test case 2
votes = ['A-0', 'B-1', 'C-1', 'D-1', 'E-0', 'F-1', 'G-1', 'H-0']
assert my_function(votes, 'H') == 0

# Test case 3
votes = ['A-1', 'B-1', 'C-1', 'D-1', 'E-1', 'F-1', 'G-1', 'H-1']
assert my_function(votes, 'A') == 1

# Test case 4
```

```python
votes = ['A-0', 'B-0', 'C-0', 'D-0', 'E-0', 'F-0', 'G-0', 'H-0']
assert my_function(votes, 'A') == 0
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```python
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

Exercise 28: Vowels

Exercise Description

You have a string `a` as input, along with two integer numbers `s` and `e`. Define a function `my_function(a, s, e)` that returns **how many vowels** are in the substring starting at position `s` and ending at position `e` (both included).

Consider the vowels `a`, `e`, `i`, `o`, `u` in both lowercase and uppercase.

Your solution

```python
def my_function(a, s, e):
    # Write your solution here
```

```
    pass
```

Test your solution

```
# Test case 1 (Example)
a = 'I study at Luiss'
assert my_function(a, 0, 15) == 5

# Test case 2
a = 'aaccccoa'
assert my_function(a, 0, 7) == 4

# Test case 3
a = 'AAAAmmm'
assert my_function(a, 4, 6) == 0

# Test case 4
a = 'xyz rtw lpq'
assert my_function(a, 1, 7) == 0
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

Exercise 29: Aggregate Address

Exercise Description

Write a function `my_function(addresses)` that aggregates addresses by town.

`addresses` is a list of tuples, where each tuple has two string elements: `(town, street_name)`. For example, it could be `addresses = [('Roma', 'Via Panama'), ('Roma', 'Viale Romania'), ('Milano', 'Corso Sempione')]`.

The function shall return a dictionary that maps each town to the number of addresses in that town from the input list. If the input list is empty, the function should return an empty dictionary.

Your solution

```
# Write your solution here
```

Test your solution

```
# Test case 1 (Example)
a = my_function([('Roma', 'Via Panama'), ('Roma', 'Viale Romania'), ('Milano', 'Corsc
assert a == {'Roma': 2, 'Milano': 1}

# Test case 2
a = my_function([('Londra', 'Via Panama'), ('Milano', 'Viale Romania'), ('Milano', '(
assert a == {'Londra': 1, 'Milano': 2}
```

```
# Test case 3
a = my_function([])
assert a == {}

# Test case 4
a = my_function([('Firenze', 'Via Panama'), ('Como', 'Viale Romania'), ('Napoli', 'Co
assert a == {'Firenze': 1, 'Como': 1, 'Napoli': 1}
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

Exercise 30: Dict Game 1

Exercise Description

Let `D` be a dictionary whose keys are words (strings) and whose corresponding values are integer numbers. Further, let `L` be a list of words.

Write a function `my_function(D, L)` that, given as input a dictionary `D` and a list `L`, returns the sum of the lengths of the words in the dictionary that satisfy the following requirements:

- The word must be present in the list
- The word must contain at least 1 uppercase character
- The value associated with the word must be an even number

If the input list is empty, the function must return 0.

Your solution

```python
# Write your solution here
```

Test your solution

```python
# Test case 1 (Example)
D = {"Sara" : 8, "Mara" : 3}
L = ["Alessio", "Sara", "Giorgio", "Mara"]
result = my_function(D, L)
assert result == 4

# Test case 2
D = {"Sara" : 8, "ilaria" : 4}
L = ["Alessio", "Sara", "Giorgio", "Mara", "ilaria"]
result = my_function(D, L)
assert result == 4

# Test case 3
D = {"Sara" : 8, "Ilaria" : 4}
L = ["Alessio", "Sara", "Giorgio", "Mara", "Ilaria"]
result = my_function(D, L)
assert result == 10
```

```python
# Test case 4
D = {"Sara" : 8, "Ilaria" : 7}
L = []
result = my_function(D, L)
assert result == 0
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```python
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

Exercise 31: Dict Game 2

Exercise Description

Let `D` be a dictionary whose keys are words (strings) and whose corresponding values are integer numbers. Further, let `L` be a list of words.

Write a function `my_function(D, L)` that, given as input a dictionary `D` and a list `L`, returns the sum of the lengths of the words in the dictionary that satisfy the following requirements:

- The word must be present in the list
- The word must contain at least 1 uppercase character
- The value associated with the word must be an even number

If the input list is empty, the function must return 0.

Your solution

```
# Write your solution here
```

Test your solution

```
# Test case 1 (Example)
D = {"Sara" : 8, "Mara" : 3}
L = ["Alessio", "Sara", "Giorgio", "Mara"]
result = my_function(D, L)
assert result == 4

# Test case 2
D = {"Sara" : 8, "ilaria" : 4}
L = ["Alessio", "Sara", "Giorgio", "Mara", "ilaria"]
result = my_function(D, L)
assert result == 4

# Test case 3
D = {"Sara" : 8, "Ilaria" : 4}
L = ["Alessio", "Sara", "Giorgio", "Mara", "Ilaria"]
result = my_function(D, L)
assert result == 10
```

```python
# Test case 4
D = {"Sara" : 8, "Ilaria" : 7}
L = []
result = my_function(D, L)
assert result == 0
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```python
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

Exercise 32: Dict Game 3

Exercise Description

Let `D` be a dictionary whose keys are words (strings) and whose corresponding values are dates. Further, let `L` be a list of words and let `A` and `B` be two `datetime.date` objects where `A < B` (or `A` comes before `B`).

Write a function `my_function(D, L, A, B)` that, given as input a dictionary `D`, a list `L` and the two `date` objects `A` and `B`, returns the number of words (keys) in the dictionary that satisfy the following requirements:

- The word must be present in the list
- The word must contain at least one uppercase letter
- The date associated with the word must be between `A` and `B`

If the input list is empty, the function must return `0`.

Hint: Remember that it is possible to use the common operators for comparison (e.g., <, >, ==) on `date` objects.

Recall: To use the `datetime` module in your code, please make sure to import it before the `def` of your function.

Your solution

```
# Write your solution here
```

Test your solution

```
# Test case 1 (Example)
import datetime
D = {
    "apple": datetime.date(2023, 3, 15),
    "Banana": datetime.date(2023, 4, 10),
    "Orange": datetime.date(2022, 12, 5),
    "Grapes": datetime.date(2023, 5, 20),
}
L = ["apple", "Banana", "Cherry", "Grapes"]
A = datetime.date(2023, 1, 1)
```

```python
B = datetime.date(2023, 6, 30)
result = my_function(D, L, A, B)
assert result == 2

# Test case 2
D = {
    "Hello": datetime.date(2023, 3, 15),
    "World": datetime.date(2023, 4, 10),
    "Python": datetime.date(2022, 12, 5),
    "Programming": datetime.date(2023, 5, 20),
}
L = ["Hello", "Programming", "AI", "MachineLearning"]
A = datetime.date(2023, 1, 1)
B = datetime.date(2023, 6, 30)
result = my_function(D, L, A, B)
assert result == 2

# Test case 3
D = {
    "Car": datetime.date(2023, 3, 15),
    "Bus": datetime.date(2023, 4, 10),
    "Bicycle": datetime.date(2022, 12, 5),
    "Train": datetime.date(2023, 5, 20),
}
L = ["Car", "Bus", "Bicycle", "Train"]
A = datetime.date(2023, 3, 1)
B = datetime.date(2023, 4, 30)
result = my_function(D, L, A, B)
assert result == 2

# Test case 4
```

```
D = {
    "Sun": datetime.date(2023, 3, 15),
    "Moon": datetime.date(2023, 4, 10),
    "Stars": datetime.date(2022, 12, 5),
    "Planets": datetime.date(2023, 5, 20),
}
L = ["Sun", "Moon", "Stars", "Planets"]
A = datetime.date(2024, 1, 1)
B = datetime.date(2024, 6, 30)
result = my_function(D, L, A, B)
assert result == 0
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

Exercise 33: Dict Game 4

Exercise Description

Let D be a dictionary whose keys are words (strings) and whose corresponding values are integer numbers. Further, let L be a list of words.

Write a function `my_function(D, L)` that, given as input a dictionary `D` and a list `L`, returns the sum of the lengths of the words in the dictionary that satisfy the following requirements:

- The word must be present in the list
- The word must contain at least 1 uppercase character
- The value associated with the word must be an even number

If the input list is empty, the function must return 0.

Your solution

```
# Write your solution here
```

Test your solution

```
# Test case 1 (Example)
D = {"Sara" : 8, "Mara" : 3}
L = ["Alessio", "Sara", "Giorgio", "Mara"]
result = my_function(D, L)
assert result == 4

# Test case 2
D = {"Sara" : 8, "ilaria" : 4}
L = ["Alessio", "Sara", "Giorgio", "Mara", "ilaria"]
result = my_function(D, L)
assert result == 4

# Test case 3
D = {"Sara" : 8, "Ilaria" : 4}
```

```
L = ["Alessio", "Sara", "Giorgio", "Mara", "Ilaria"]
result = my_function(D, L)
assert result == 10

# Test case 4
D = {"Sara" : 8, "Ilaria" : 7}
L = []
result = my_function(D, L)
assert result == 0
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

Exercise 34: Dict Game 5

Exercise Description

Let `D` be a dictionary whose keys are words (strings) and whose corresponding values are integer numbers. Further, let `L` be a list of words.

Write a function `my_function(D, L)` that, given as input a dictionary `D` and a list `L`, returns the sum of the lengths of the words in the dictionary that satisfy the following requirements:

- The word must be present in the list
- The word must contain at least 1 uppercase character
- The value associated with the word must be an even number

If the input list is empty, the function must return 0.

Your solution

```python
# Write your solution here
```

Test your solution

```python
# Test case 1 (Example)
D = {"Sara" : 8, "Mara" : 3}
L = ["Alessio", "Sara", "Giorgio", "Mara"]
result = my_function(D, L)
assert result == 4

# Test case 2
D = {"Sara" : 8, "ilaria" : 4}
L = ["Alessio", "Sara", "Giorgio", "Mara", "ilaria"]
result = my_function(D, L)
assert result == 4

# Test case 3
D = {"Sara" : 8, "Ilaria" : 4}
```

```
L = ["Alessio", "Sara", "Giorgio", "Mara", "Ilaria"]
result = my_function(D, L)
assert result == 10

# Test case 4
D = {"Sara" : 8, "Ilaria" : 7}
L = []
result = my_function(D, L)
assert result == 0
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

Exercise 35: Encrypt

Exercise Description

Define a function `my_function(S, D)` that decrypts a text `S`, previously encrypted using a simple cipher called "letter substitution cipher". `D` is a dictionary that maps encrypted characters to decrypted characters. If a character of string `S` is a key of the dictionary, it shall be substituted by its corresponding dictionary value, otherwise it should not change. The function shall return the decrypted text.

Your solution

```python
# Write your solution here
```

Test your solution

```python
# Test case 1 (Example)
a = my_function("hvssm", {"v": "e", "t": "w", "s": "l", "m": "o"})
assert a == "hello"

# Test case 2
a = my_function("vtmx", {"v": "e", "t": "w", "s": "l", "m": "o"})
assert a == "ewox"

# Test case 3
a = my_function("xxxxqqqq", {"v": "e", "t": "w", "s": "l", "m": "o"})
assert a == "xxxxqqqq"

# Test case 4
a = my_function("", {"v": "e", "t": "w", "s": "l", "m": "o"})
assert a == ""
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code**

before running the cell containing the Python Tutor link.

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

Exercise 36: Football League

Exercise Description

Write a function `my_function(matches)` that computes the points of the teams taking part ir a football league.

Parameter `matches` is a list of tuples representing matches, such as: `('Roma', 'Juventus', 3, 2)`.

Each tuple has the following four elements:

1. the name of the home team
2. the name of the away team
3. the number of goals of the home team
4. the number of goals of the away team

The function must compute a dictionary that maps each team to its total number of points in the league, where teams are awarded 3 points for a win, 1 point for a draw and no points for a loss. The function shall then return the difference between the maximum number of points and the minimum number of points achieved by teams that took part in the league.

Back-of-the-envelope example: if `matches` is

`[('Roma', 'Juventus', 3, 2), ('Milan', 'Juventus', 1, 1), ('Roma', 'Milan', 5, 1), ('Juventus', 'Milan', 2, 1)]`

- Roma gets in total 6 points, i.e., 3 points from each of its two matches.
- Juventus gets in total 4 points: 1 point from the draw with Milan, and 3 points from the winning match with Milan.
- Milan gets only 1 point from the draw with Juventus.

The function shall thus return 5 = 6 - 1.

Your solution

```python
# Write your solution here
```

Test your solution

```python
# Test case 1 (Example)
matches = [('Roma', 'Juventus', 3, 2), ('Milan', 'Juventus', 1, 1), ('Roma', 'Milan',
assert my_function(matches) == 5

# Test case 2
matches = [('Roma', 'Juventus', 3, 3)]
assert my_function(matches) == 0

# Test case 3
matches = [('Roma', 'Juventus', 3, 3), ('Milan', 'Juventus', 2, 1)]
```

```python
assert my_function(matches) == 2

# Test case 4
matches = [
    ('Barcelona', 'Real Madrid', 2, 2),
    ('Manchester United', 'Manchester City', 1, 3),
    ('Liverpool', 'Chelsea', 2, 1),
    ('Bayern Munich', 'Borussia Dortmund', 3, 2),
    ('Paris Saint-Germain', 'AS Monaco', 1, 1),
    ('Atletico Madrid', 'Sevilla', 0, 2),
    ('AC Milan', 'Inter Milan', 2, 2),
    ('Juventus', 'Napoli', 1, 0),
    ('Ajax', 'PSV Eindhoven', 3, 1),
    ('Porto', 'Benfica', 2, 2)
]
assert my_function(matches) == 3
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```python
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

Exercise 37: Morse Code

Exercise Description

Define a function `my_function(s, morse_dict)` that decodes a string `s`, representing a word encoded in Morse Code.

In Morse Code:

- each letter is encoded as a sequence of dashes and dots. For example, letter `C` is encoded as `-.-.` (dash dot dash dot)
- encoded letter are separated by a `/` symbol (slash)

The function shall return the decoded string or an empty string in case `s` is empty.

Parameter `morse_dict` is a dictionary whose keys are morse-encoded letters: keys are mapped to plaintext (decoded) letters.

Your solution

```
# Write your solution here
```

Test your solution

```
# Test case 1 (Example)
morse_code = {
    ".-": "A",
    "-...": "B",
    "-.-.": "C",
    "-..": "D",
```

```
    ".": "E",
    "..-.": "F",
    "--.": "G",
}
word_in_morse = "..-./.-/-.-./."
word = my_function(word_in_morse, morse_code)
assert word == "FACE"

# Test case 2
morse_code = {
    ".-": "A",
    "-...": "B",
    "-.-.": "C",
    "-..": "D",
    ".": "E",
    "..-.": "F",
    "--.": "G",
}
word_in_morse = "--./../--./..-."
word = my_function(word_in_morse, morse_code)
assert word == "GEGF"

# Test case 3
morse_code = {
    ".-": "A",
    "-...": "B",
    "-.-.": "C",
    "-..": "D",
    ".": "E",
    "..-.": "F",
    "--.": "G",
```

```
}
word_in_morse = ""
word = my_function(word_in_morse, morse_code)
assert word == ""

# Test case 4
morse_code = {
    ".-": "A",
    "-...": "B",
    "-.-.": "C",
    "-..": "D",
    ".": "E",
    "..-.": "F",
    "--.": "G",
}
word_in_morse = "-.-./-.-./-.-./-.-./-.-./-.-."
word = my_function(word_in_morse, morse_code)
assert word == "CCCCCC"
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

Exercise 38: Musical Albums

Exercise Description

Write a Python function called `my_function(L, S)` that takes as input a list `L` and a string `S`.

List `L` is a list of musical albums, where each album is described as a dictionary with 5 fields:

- `"title"` (string): the title of the album
- `"artist"` (string): the artistIf there are no albums with such genre in `L`, the function must return `-1`. If `L` is empty, the function must return `0`.
- `"genre"` (string): its genre
- `"rating"` (float): the rating for the album (scale from 1 to 5)The function must return the average rating calculated amongst all albums in `L` whose genre is `S`.
- `"year"` (integer): the release year

whereas `S` is a string which corresponds to a genre.

Your solution

```
# Write your solution here
```

Test your solution

```
# Test case 1 (Example)
S = "Grunge"
L = [
    {
        "title": "Nevermind",
        "artist": "Nirvana",
        "genre": "Grunge",
        "year": 1991,
        "rating": 4.5,
    },
    {
        "title": "London Calling",
        "artist": "The Clash",
        "genre": "Punk",
        "year": 1979,
        "rating": 4.7,
    },
    {
        "title": "Unknown Pleasures",
        "artist": "Joy Division",
        "genre": "Dark",
        "year": 1979,
        "rating": 4.6,
    },
    {
        "title": "Siamese Dream",
        "artist": "The Smashing Pumpkins",
        "genre": "Grunge",
        "year": 1993,
```

```python
        "rating": 4.4,
    },
]
assert my_function(L, S) == 4.45

# Test case 2
S = "Parento"
L = [
    {
        "title": "Nevermind",
        "artist": "Nirvana",
        "genre": "Grunge",
        "year": 1991,
        "rating": 4.5,
    },
    {
        "title": "London Calling",
        "artist": "The Clash",
        "genre": "Punk",
        "year": 1979,
        "rating": 4.7,
    },
    {
        "title": "Unknown Pleasures",
        "artist": "Joy Division",
        "genre": "Dark",
        "year": 1979,
        "rating": 4.6,
    },
    {
        "title": "Siamese Dream",
```

```python
        "artist": "The Smashing Pumpkins",
        "genre": "Grunge",
        "year": 1993,
        "rating": 4.4,
    },
]
assert my_function(L, S) == -1

# Test case 3
S = "Dark"
L = [
    {
        "title": "Nevermind",
        "artist": "Nirvana",
        "genre": "Grunge",
        "year": 1991,
        "rating": 4.5,
    },
    {
        "title": "London Calling",
        "artist": "The Clash",
        "genre": "Punk",
        "year": 1979,
        "rating": 4.7,
    },
    {
        "title": "Unknown Pleasures",
        "artist": "Joy Division",
        "genre": "Dark",
        "year": 1979,
        "rating": 4.6,
```

```
    },
    {
        "title": "Siamese Dream",
        "artist": "The Smashing Pumpkins",
        "genre": "Grunge",
        "year": 1993,
        "rating": 4.4,
    },
]
assert my_function(L, S) == 4.6

# Test case 4
S = "Rock"
L = []
assert my_function(L, S) == 0
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

Exercise 39: Planet 4z Mission Cost

Exercise Description

4z is a pacific planet where all of its inhabitants live peacefully. The only rule they must obey is that their names must contain the letter `'z'` or they have to be at most 4 letters long. It's 11-11-2222 when planet Earth, close to extinction, sends an SOS to planet 4z. Attached to the SOS, there's a dictionary of earthlings whose structure is:

```
{
"name of earthling 1" : (its age, its weight),
"name of earthling 2" : (its age, its weight),
...
}
```

Help the aliens from planet 4z to calculate the overall cost of the rescue by writing a function `my_function(humans)` that, given the dictionary of earthlings, returns the mission cost.

The cost of a space shuttle for a human whose weight is more than 80kg is 25eth, otherwise it is 15eth. Furthermore, for humans older than 80 years old, there is an additional 5eth charge for carrying medical equipment.

Your solution

```
# Write your solution here
```

Test your solution

```
# Test case 1 (Example)
a = {'giorgio' : (35, 88), 'ind' : (20, 15), 'alessioz' : (90, 67) }
assert my_function(a) == 35

# Test case 2
```

```
a = {'giorgioz' : (35, 88), 'ind' : (20, 15), 'alessioz' : (90, 67) }
assert my_function(a) == 60

# Test case 3
a = {'giorgio' : (35, 88), 'indro' : (20, 15), 'alessio' : (90, 67) }
assert my_function(a) == 0

# Test case 4
a = { }
assert my_function(a) == 0
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

Exercise 40: Shopping By Quantity

Exercise Description

Shopping by Quantity

Write a function `my_function(prices, cart)` that computes the total price to be paid for items in a shopping cart.

Parameter `prices` is a list of tuples `(item, price)`, where `item` is a string and `price` is a number: for example, `prices` could be `[('cookies', 4.2), ('pasta', 3.0), ('eggs', 3.2), ('milk', 2.3)]`.

Parameter `cart` is a dictionary that maps items to the number of sold units, for example `{'pasta': 4, 'milk': 2, 'cookies': 3}`.

You can assume that each key in the cart dictionary appears in the prices list.

Back-of-the-envelope example: `my_function([('cookies', 4.2), ('pasta', 3.0), ('eggs', 3.2), ('milk', 2.3)], {'pasta': 4, 'milk': 2, 'cookies': 3})` should return 29.2 because $29.2 = 4.2 \times 3 + 3.0 \times 4 + 2.3 \times 2$.

The function shall return the total cost of items in the cart dictionary. If there are no items in the cart, the function should return 0.

Your solution

```
# Write your solution here
```

```
Failed to start the Kernel.

RuntimeError: There is no current event loop in thread 'MainThread'.

View Jupyter <a href='command:jupyter.viewOutput'>log</a> for further detai
ls.
```

Test your solution

```
# Test case 1 (Example)
a = my_function([('cookies', 4.2), ('pasta', 3.0), ('eggs', 3.2), ('milk', 2.3)], {'p
assert abs(a - 29.2) < 0.0001

# Test case 2
a = my_function([('cookies', 4.2), ('orange', 1.0), ('eggs', 3.2), ('milk', 2.3)], {'
assert abs(a - 8.6) < 0.0001

# Test case 3
a = my_function([('cookies', 4.2), ('orange', 1.0), ('bacon', 3.2), ('milk', 2.3)], {
assert abs(a - 35.2) < 0.0001

# Test case 4
a = my_function([('cookies', 4.2), ('orange', 1.0), ('bacon', 3.2), ('milk', 2.3)], {
assert abs(a - 0.0) < 0.0001
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

Exercise 41: Shopping By Prices

Exercise Description

Shopping by Prices

Write a function `my_function(shopping_cart, prices)` that computes the total price to be paid for items in a shopping cart.

Parameter `shopping_cart` is a list of tuples `(item, quantity)`, where `item` is a string and `quantity` is a number: for example, `shopping_cart` could be `[('pasta', 3), ('milk', 2), ('cookies', 4)]`.

Parameter `prices` is a dictionary that maps items to their unit price, for example `{'pasta': 0.69, 'milk': 1.19, 'eggs': 0.79, 'bread': 1.99, 'cookies': 2.49}`.

You can assume that each item in the shopping list is a key in the `prices` dictionary.

Back-of-the-envelope example: `my_function([('pasta', 3), ('milk', 2), ('cookies', 4)], {'pasta': 0.69, 'milk': 1.19, 'eggs': 0.79, 'bread': 1.99, 'cookies': 2.49})` should return 14.41 because $14.41 = 0.69 \times 3 + 1.19 \times 2 + 2.49 \times 4$.

The function shall return the total cost of items in the shopping cart. If there are no items in the cart, the function should return `0`.

Your solution

```python
# Write your solution here
```

Test your solution

```python
# Test case 1 (Example)
a = my_function([('pasta', 3), ('milk', 2), ('cookies', 4)], {'pasta': 0.69, 'milk':
assert a == 14.41

# Test case 2
a = my_function([('orange', 1), ('cookies', 10)], {'cookies': 0.69, 'milk': 1.19, 'eg
assert a == 9.39

# Test case 3
a = my_function([('orange', 1)], {'cookies': 0.69, 'milk': 1.19, 'eggs': 0.79, 'bread
assert a == 2.49

# Test case 4
a = my_function([], {'cookies': 0.69, 'milk': 1.19, 'eggs': 0.79, 'bread': 1.99, 'ora
assert float(a) == 0.0
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```python
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

Exercise 42: Wallet

Exercise Description

Write a function `my_function(wallet, price)` that computes the total value inside the wallet in Euro.

Parameter `wallet` is a list of tuples `(currency, quantity)`, where `currency` is a string and `quantity` is a number: for example, `wallet` could be `[("gbp", 3), ("dollar", 2), ("gbp", 4)]`.

Parameter `price` is a dictionary that maps the conversion of different currencies in euro (1 euro) , for example `{"krona": 7.44, "dollar": 1.22, "gbp": 0.89, "real": 6.65}`.

You can assume that the currency in each item in the wallet appears as a key in the price dictionary.

The function shall return the total value in euro inside the wallet. If there are no items in the wallet, the function should return 0.

Back-of-the-envelope example: `my_function([("gbp", 2), ("dollar", 2)], {"krona": 7.44, "dollar": 1.22, "gbp": 0.89, "real": 6.65})` should return $4.22 = 2 \times 0.89 + 2 \times 1.22$.

Your solution

```
# Write your solution here
```

Test your solution

```
# Test case 1 (Example)
a = my_function([("gbp", 2), ("dollar", 2)], {"krona": 7.44, "dollar": 1.22, "gbp": 0
assert a == 4.22

# Test case 2
a = my_function([("dollar", 2)], {"krona": 7.44, "dollar": 1.22, "gbp": 0.89, "real":
assert a == 2.44

# Test case 3
a = my_function([("krona", 1), ("dollar", 2)], {"krona": 7.44, "dollar": 1.22, "gbp":
assert a == 9.88

# Test case 4
a = my_function([], {"krona": 7.44, "dollar": 1.22, "gbp": 0.89, "real": 6.65})
assert a == 0
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

Exercise 43: Automatic writing

Exercise Description

Write a function `my_function(a, b)` that takes two lists:

- `a` is a list of characters, for example `['b', 'e', 'l', 'a']`
- `b` is a list of integers, for example `[1, 1, 2, 1]`

The function should behave as follows:

- If the two lists `a` and `b` have different lengths, the function returns `-1`.
- If there is at least one negative value in `b`, the function returns `-1`.
- Otherwise, the function returns a string containing the characters in `a` repeated as many times as the number specified in the corresponding position in `b`.

For example, `my_function(['b','e','l','a'], [1,1,2,1])` should return `'bella'`.

Your solution

```
def my_function(a, b):
    # Write your solution here
    pass
```

Test your solution

```python
# Test case 1 (Example)
assert my_function(['b','e','l','a'], [1,1,2,1]) == 'bella'

# Test case 2
assert my_function(['a','b','c'], [3,2,0]) == 'aaabb'

# Test case 3
assert my_function(['a','b','c'], [3,2,-1]) == -1

# Test case 4
assert my_function(['a','b','c'], [3,2,2,2]) == -1
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```python
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

Exercise 44: List concatenations even n

Exercise Description

Write a function `my_function(lst, n)` which, given as input a list of strings `lst` and a number `n`, returns a string that concatenates all the strings in the list that:

- have length strictly greater than `n`, and
- have an even length.

If none of the strings satisfy the requirements, the function must return the string `'no'`.

If the input list is empty, the function must return `-1`.

Your solution

```
def my_function(lst, n):
    # Write your solution here
    pass
```

Test your solution

```
# Test case 1 (Example)
assert my_function(["chirus", "sam", "z", "alessio martino"], 4) == "chirus"

# Test case 2
assert my_function(["chirus", "samo", "zo", "alessio martino", "time"], 2) == "chirus

# Test case 3
assert my_function(["chirus", "samo", "zo", "alessio martino", "time"], 22) == "no"

# Test case 4
assert my_function([], 4) == -1
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

Exercise 45: Loop Game 1: prime-weighted sum

Exercise Description

Write a function `my_function(lst)` which takes as input a list of numbers `lst` and returns the sum of the numbers in the list, with the following rules:

- Prime numbers are worth double (for example, 7 is worth 14).
- If a `0` is encountered, the current sum is reset to `0`.
- If the number `-1` is encountered, the loop is interrupted and the function returns the sum obtained up to that point.

If the list is empty, the function should return `0`.

Your solution

```python
def my_function(lst):
    # Write your solution here
    pass
```

Test your solution

```python
# Test case 1 (Example)
assert my_function([4, 6, 8]) == 18

# Test case 2
assert my_function([99, 45, 18, 0, 7]) == 14

# Test case 3
assert my_function([40, 40, -1, 100, 7, 0, 12]) == 80

# Test case 4
assert my_function([]) == 0
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```python
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

Exercise 46: Loop Game 2: character counter

Exercise Description

Write a function `my_function(lst)` which takes as input a list of strings `lst` and returns the total "value" of all the characters in the list. The rules are:

- Lowercase letters are worth 1 point each.
- Uppercase letters are worth 2 points each.
- If the character `'0'` is encountered, the current total is doubled (the character `'0'` itself does not add points).
- If the lowercase character `'z'` is encountered, the loop stops immediately and the function returns `-1`.

The function returns the final value according to these rules.

Your solution

```
def my_function(lst):
    # Write your solution here
    pass
```

Test your solution

```
# Test case 1 (Example)
assert my_function(["hello", "test", "hi"]) == 11
```

```
# Test case 2
assert my_function(["hEllo", "teSt", "hi"]) == 13

# Test case 3
assert my_function(["hEllo", "teSt", "0i"]) == 23

# Test case 4
assert my_function(["hEllz", "teSt", "0i"]) == -1
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

Exercise 47: Loop Game 3: even numbers and powers

Exercise Description

Write a function `my_function(lst)` which takes as input a list of numbers `lst` and returns the sum of the numbers in the list, with the following rules:

- Even numbers less than 100 are worth double (for example, 50 is worth 100).
- If a `0` is encountered, the current sum is doubled.
- If the number `-1` is encountered, the loop is interrupted and the function returns the square of the current sum (`sum ** 2`).

If the list is empty, the function should return `0`.

Your solution

```
def my_function(lst):
    # Write your solution here
    pass
```

Test your solution

```
# Test case 1 (Example)
assert int(my_function([1, 1, 1, 102])) == 105

# Test case 2
assert int(my_function([2, 2, 2, 102])) == 114

# Test case 3
assert int(my_function([])) == 0

# Test case 4
assert int(my_function([2, 2, -1, 102])) == 64
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

Exercise 48: Loop Game 4: filtered strings count

Exercise Description

Write a function `my_function(lst, n)` which takes as inputs a list of strings `lst` and an integer `n` and returns the number of elements in the list that satisfy all the following conditions:

- the string length is **less than or equal to** `n`, and
- the string does **not** contain any uppercase characters.

If an empty string `""` is encountered in the list, the function must stop and return the count obtained up to that point.

Your solution

```
def my_function(lst, n):
    # Write your solution here
    pass
```

Test your solution

```
# Test case 1 (Example)
assert my_function(["a", "ab", "abc"], 3) == 3

# Test case 2
assert my_function(["a", "ab", "abc"], 2) == 2

# Test case 3
assert my_function(["A", "aB", "abc"], 3) == 1

# Test case 4
assert my_function(["abcd", "ab", "A", "", "abc"], 3) == 1
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

Exercise 49: Merge words

Exercise Description

Write a function `my_function(a, b)` that adds two lists index-wise.

- `a` and `b` are lists whose elements are strings (or characters).
- The function returns a new list where each element is the concatenation of the elements from `a` and `b` in the same position.

If the lists do not have the same length, the function should return `-1`.

Your solution

```
def my_function(a, b):
    # Write your solution here
    pass
```

Test your solution

```
# Test case 1 (Example)
assert my_function(['Pro','Ale','Mar'], ['f','ssio','tino']) == ['Prof', 'Alessio', '

# Test case 2
assert my_function(['Pro','Ale','Mar'], ['f','ssio']) == -1

# Test case 3
assert my_function(['Ta','va','gat'], ['nto',' la','ta']) == ['Tanto', 'va la', 'gatt

# Test case 4
assert my_function(['Pro','Ale'], ['nto','ssandro']) == ['Pronto', 'Alessandro']
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

Exercise 50: Password Encoder

Exercise Description

Write a function `my_function(s, letter)` that helps to build a password.

- `s` is a string.
- `letter` is a character and must be one of `['i', 'a', 's']`.

The function converts the string into a password with the following rules:

- If there are **no capital letters** in `s`, the function returns `-1`.
- If `letter` is **not** one of `'i'`, `'a'`, `'s'`, the function returns `-1`.
- If `letter` does not appear in `s`, the function returns `-1`.
- Otherwise, it returns a new string where every occurrence of `letter` is replaced by a special character according to this mapping:
    - `'i'` becomes `'!'`
    - `'a'` becomes `'@'`
    - `'s'` becomes `'$'`

All other characters remain unchanged.

Your solution

```python
def my_function(s, letter):
    # Write your solution here
    pass
```

Test your solution

```python
# Test case 1 (Example)
assert my_function("Mammabuttalapasta", "a") == "M@mm@butt@l@p@st@"

# Test case 2
assert my_function("SerpentiSuonatoridiSonagli", "s") == -1

# Test case 3
assert my_function("CaniCattivi", "i") == "Can!Catt!v!"
```

```
# Test case 4
assert my_function("CaniCattivi", "q") == -1
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

Exercise 51: Planet 4z

Exercise Description

It is the year 2222 and the Earth is nearing its end. The inhabitants of planet 4z have decided to help the earthlings by sending escape ships. Each spacecraft can hold only one person.

On planet 4z there is an inviolable rule: the name of all its inhabitants must **contain the letter `'z'`** or be **at most 4 characters long**. Any humans who fail to meet these requirements will not be saved.

Write a function `my_function(inhabitants)` that, given as input a list of names of earthlings, returns the number of ships to be sent to planet Earth (i.e., the number of humans that satisfy the rule).

If none of the names meet the requirements, the function must return `-1`.

Your solution

```python
def my_function(inhabitants):
    # Write your solution here
    pass
```

Test your solution

```python
# Test case 1 (Example)
assert my_function(["chirus", "zzzzzzz"]) == 1

# Test case 2
assert my_function(["chirus"]) == -1

# Test case 3
assert my_function(["chirus", "sam", "z", "alessio martino"]) == 2

# Test case 4
assert my_function([]) == -1
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

Exercise 52: Repeat

Exercise Description

Write a function `my_function(lst)` that takes a list of integers as input and returns a new list of integers.

The new list is built as follows:

- the element in position 0 is repeated 1 time,
- the element in position 1 is repeated 2 times,
- the element in position 2 is repeated 3 times, and so on.

However, you do **not** like the number `4`, therefore:

- you must **ignore** any element whose value is `4` in the output list, and
- you must **never** repeat an element **4 times** (i.e., when the position index + 1 is 4, skip that element).

Your solution

```python
def my_function(lst):
    # Write your solution here
    pass
```

Test your solution

```python
# Test case 1 (Example)
assert my_function([1, 2, 3, 4]) == [1, 2, 2, 3, 3, 3]

# Test case 2
assert my_function([9, 8, 7, 6, 5]) == [9, 8, 8, 7, 7, 7, 5, 5, 5, 5, 5]

# Test case 3
assert my_function([4, 4, 4, 2, 1]) == [1, 1, 1, 1, 1]

# Test case 4
assert my_function([4, 4, 4, 4, 4, 4, 4]) == []
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```python
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

Exercise 53: Reverse (almost) everything

Exercise Description

Write a function `my_function(lst, key)` that takes as input a list of words `lst` and a word `key`.

- If `key` is **not** in the list, the function should return `-1`.
- Otherwise, it should return a new list where **all words are reversed**, except for `key`, which remains unchanged (but stays in its original position).

Your solution

```
def my_function(lst, key):
    # Write your solution here
    pass
```

Test your solution

```
# Test case 1 (Example)
assert my_function(['hello', 'world', 'python'], 'world') == ['olleh', 'world', 'noht

# Test case 2: key not present
assert my_function(['a', 'b', 'c'], 'x') == -1

# Test case 3: key at the beginning
assert my_function(['key', 'abc', 'def'], 'key') == ['key', 'cba', 'fed']
```

```
# Test case 4: key at the end
assert my_function(['abc', 'def', 'key'], 'key') == ['cba', 'fed', 'key']
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

Exercise 54: Sandwich

Exercise Description

Write a function `my_function(lst, k)` that returns a list of all the elements in `lst` that are **between two values** `k`.

That is, for each element in the list, you check whether the element before it and the element after it are both equal to `k`. If so, this element is included in the output.

Additional rules:

- If there are multiple occurrences of the same value that satisfy the condition, the output list should contain each such value **only once** (no duplicates).
- If `k` is not in `lst`, the function should return `-1`.

Your solution

```
def my_function(lst, k):
    # Write your solution here
    pass
```

Test your solution

```
# Test case 1 (Example)
assert my_function([2, 1, 2, 3, 4, 5, 2, 4, 2, 4, 2], 2) == [1, 4]

# Test case 2: k not present
assert my_function([2, 1, 2, 3, 4, 5, 2, 4, 2, 4, 2], 7) == -1

# Test case 3: repeated values
assert my_function([2, 2, 2, 3, 4, 5, 2, 4, 2, 4, 2], 2) == [2, 4]

# Test case 4
assert my_function([2, 2, 2, 2, 2, 2, 2, 3, 4, 3, 4, 3, 2, 3, 2], 2) == [2, 3]
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code**

before running the cell containing the Python Tutor link.

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

Exercise 55: Converting Time 1

Exercise Description

Write a function `my_function(K, L)` that is given an integer number `K` and a list of tuples `L`. Each tuple is a pair consisting of an integer number and a character `'m'` (minutes), `'h'` (hours), or `'s'` (seconds).

After converting hours and minutes to seconds, the function should return the position `i` of the tuple in list `L` such that the sum of seconds up to position `i` is **smaller than or equal to** `K`. Positions start from 0.

If the input list is empty or the number of seconds specified by the tuple in position 0 is larger than `K`, the function should return `-1`.

Example: `my_function(1000, [(2, 'm'), (25, 's'), (1, 'h'), (1, 's')])` should return 1, because 2 minutes = 120 seconds, 1 hour = 3600 seconds, and 120 + 25 = 145 < 1000, while 120 + 25 + 3600 = 3745 > 1000.

Your solution

```python
def my_function(K, L):
    # Write your solution here
    pass
```

Test your solution

```python
# Test case 1 (Example)
assert my_function(1000, [(2, 'm'), (25, 's'), (1, 'h'), (1, 's')]) == 1

# Test case 2
assert my_function(121, [(2, 'm'), (25, 's'), (1, 'h'), (1, 's')]) == 0

# Test case 3
assert my_function(1, [(1, 'm'), (25, 's'), (1, 'h'), (1, 's')]) == -1

# Test case 4
assert my_function(1, []) == -1
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```python
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

Exercise 56: Converting Time 2

Exercise Description

Write a function `my_function(K, L)` that is given an integer number `K` and a list of tuples `L`. Each tuple is a triple consisting of three integer numbers, corresponding to **hours, minutes, and seconds**.

After computing the total number of seconds given by each tuple, the function should return the position `i` of the tuple in list `L` such that the sum of seconds up to position `i` is **smaller than or equal to** `K`. Positions start from 0.

If the input list is empty or the number of seconds specified by the tuple in position 0 is larger than `K`, the function should return `-1`.

Example: `my_function(4000, [(0, 2, 3), (1, 3, 5), (2, 0, 32)])` should return `1`.

Your solution

```
def my_function(K, L):
    # Write your solution here
    pass
```

Test your solution

```
# Test case 1 (Example)
assert my_function(4000, [(0, 2, 3), (1, 3, 5), (2, 0, 32)]) == 1
```

```python
# Test case 2
assert my_function(344000, [(0, 0, 3), (0, 0, 5), (2, 0, 32)]) == 2

# Test case 3
assert my_function(2, [(0, 0, 3), (1, 0, 5), (2, 0, 32)]) == -1

# Test case 4
assert my_function(2, []) == -1
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```python
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

Exercise 57: Divisible by 4

Exercise Description

You have to compute how many numbers divisible by 4 with remainder 1 or 2 you have to sum up until the value of the sum becomes **larger than or equal** to `n`.

The series of numbers divisible by 4 with remainder 1 or 2 starts with 1, 2, 5, 6, 9, 10, ... (since dividing 1 by 4 gives remainder 1, 2 gives remainder 2, 5 gives remainder 1, and so on).

Write a function `my_function(n)` that gets an integer value `n` as input and returns the **number of terms** in the smallest sum whose value is $\geq$ `n`. The sum of 0 terms is equal to 0.

Example: if the input is 25, the output is 6 because $1 + 2 + 5 + 6 + 9 = 23 < 25$ but $1 + 2 + 5 + 6 + 9 + 10 = 33 \geq 25$.

Your solution

```
def my_function(n):
    # Write your solution here
    pass
```

Test your solution

```
# Test case 1 (Example)
assert my_function(25) == 6

# Test case 2
assert my_function(8) == 3

# Test case 3
assert my_function(75) == 9

# Test case 4
assert my_function(0) == 0
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

Exercise 58: Fitting sum

Exercise Description

You are given an integer number `n` as input, representing the maximum amount available. Sum all the natural numbers (1, 2, 3, ...) that fit in `n`, i.e., all natural numbers whose cumulative sum is smaller than or equal to `n`.

Define a function `last_fit(n)` that returns the **last number** that fits in this sum.

Example: if `n = 12`, the output is 4 because `1 + 2 + 3 + 4 <= 12` but `1 + 2 + 3 + 4 + 5 > 12`.

Your solution

```
def last_fit(n):
    # Write your solution here
    pass
```

Test your solution

```
# Test case 1 (Example)
assert last_fit(12) == 4

# Test case 2
assert last_fit(8) == 3

# Test case 3
assert last_fit(10) == 4

# Test case 4
assert last_fit(100) == 13
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

Exercise 59: KM CM to M

Exercise Description

Write a function `my_function(S, L)` that is given an integer number `S` and a list of tuples `L`. Each tuple is a pair consisting of an integer number and a character `'k'` (kilometers), `'c'` (centimeters), or `'m'` (meters).

After converting kilometers and centimeters to meters, the function should return the position `i` of the tuple in list `L` such that the sum of meters up to position `i` is **smaller than or equal to** `S`. Positions start from 0.

If the input list is empty or the number of meters specified by the tuple in position 0 is larger than `S`, the function should return `-1`.

Example: `my_function(1000, [(2, 'm'), (30000, 'c'), (5, 'k')])` should return 1, because 30000 cm = 300 m, 5 km = 5000 m, and 2 + 300 = 302 ≤ 1000, while 2 + 300 + 5000 = 5302 > 1000.

Your solution

```
def my_function(S, L):
    # Write your solution here
    pass
```

Test your solution

```
# Test case 1 (Example)
assert my_function(1000, [(2, 'm'), (30000, 'c'), (5, 'k')]) == 1

# Test case 2
assert my_function(1000, [(2, 'm'), (30000, 'c'), (5, 'c')]) == 2
```

```
# Test case 3
assert my_function(1, [(20, 'k'), (30000, 'c'), (5, 'k')]) == -1

# Test case 4
assert my_function(1000, []) == -1
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

Exercise 60: Larger in a sequence

Exercise Description

Write a function `first_larger(a, b, n)` that receives three natural numbers `a`, `b`, and `n`.

Starting from `a` and `b`, the function generates a sequence where each element is equal to the sum of the previous two elements. It then returns the **first element in the sequence that is strictly larger than** `n`.

Example: if `a = 1`, `b = 2`, and `n = 50`, the sequence is 1, 2, 3, 5, 8, 13, 21, 34, 55, ... and the function should return 55.

Your solution

```
def first_larger(a, b, n):
    # Write your solution here
    pass
```

Test your solution

```
# Test case 1 (Example)
assert first_larger(1, 2, 50) == 55

# Test case 2
assert first_larger(1, 2, 15) == 21

# Test case 3
a = 2; b = 3; n = 8
assert first_larger(a, b, n) == 8

# Test case 4
a = 1; b = 1; n = 1000
assert first_larger(a, b, n) == 1597
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code**

before running the cell containing the Python Tutor link.

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

Exercise 61: Sum 2 Series

Exercise Description

You have to compute the sum of a series of numbers 2, 22, 222, 2222, and so on, until the value of the sum becomes larger than or equal to `n`.

Write a function `my_function(n)` that gets an integer value `n` as input parameter and returns the **number of terms** in the smallest sum whose value is ≥ `n`. The sum of 0 terms is equal to 0.

Hint: each number can be obtained from the previous one in the series by multiplying it by 10 and adding 2 (e.g., 2222 = 222 * 10 + 2).

Example: if the input is 25, the output is 3 because 2 + 22 = 24 < 25 but 2 + 22 + 222 = 246 ≥ 25.

Your solution

```python
def my_function(n):
    # Write your solution here
    pass
```

Test your solution

```python
# Test case 1 (Example)
assert my_function(25) == 3

# Test case 2
assert my_function(1000) == 4

# Test case 3
assert my_function(2468) == 4

# Test case 4
assert my_function(1) == 1
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```python
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

Exercise 62: Sum Fermat

Exercise Description

You have to compute the sum of numbers 11, 101, 1001, 10001, 100001, and so on, until the value of the sum becomes larger than or equal to `n`.

Write a function `my_function(n)` that gets an integer value `n` as input parameter and returns the **number of terms** in the smallest sum whose value is ≥ `n`. The sum of 0 terms is equal to 0.

Hint: each number in the series is a power of 10 plus 1 (e.g., 10001 = 10^4 + 1).

Example: if the input is 250, then the output is 3 because 11 + 101 = 112 < 250 but 11 + 101 + 1001 = 1113 ≥ 250.

Your solution

```python
def my_function(n):
    # Write your solution here
    pass
```

Test your solution

```python
# Test case 1 (Example)
assert my_function(250) == 3

# Test case 2
```

```python
assert my_function(11114) == 4

# Test case 3
assert my_function(3000) == 4

# Test case 4
assert my_function(1) == 1
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```python
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

Exercise 63: Sum Km and Cm to M

Exercise Description

Write a function `my_function(S, L)` that is given an integer number `S` and a list of tuples `L`. Each tuple is a pair consisting of an integer number and a character `'k'` (kilometers), `'c'` (centimeters), or `'m'` (meters).

After converting kilometers and centimeters to meters, the function should return the **largest sum** `s` of meters such that `s <= S`. The sum must be computed starting from position 0 and using elements in consecutive positions in the list.

If the input list is empty or the number of meters specified by the tuple in position 0 is larger than `S`, the function should return `-1`.

Example: `my_function(1000, [(2, 'm'), (30000, 'c'), (5, 'k')])` should return 302, because 30000 cm = 300 m, 5 km = 5000 m, and 2 + 300 = 302 ≤ 1000, while 2 + 300 + 5000 = 5302 > 1000.

Your solution

```python
def my_function(S, L):
    # Write your solution here
    pass
```

Test your solution

```python
# Test case 1 (Example)
assert int(my_function(1000, [(2, 'm'), (30000, 'c'), (5, 'k')])) == 302

# Test case 2
assert int(my_function(1000, [(2, 'm'), (3, 'm'), (5, 'm')])) == 10

# Test case 3
assert int(my_function(1, [(2, 'm'), (30000, 'c'), (5, 'k')])) == -1

# Test case 4
assert int(my_function(1000, [])) == -1
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

Exercise 64: Sum of Triangular

Exercise Description

You have to compute the sum of triangular numbers 1, 3, 6, 10, 15, 21, ... until the value of the sum becomes larger than or equal to `n`.

Write a function `my_function(n)` that gets an integer value `n` as input parameter and returns the **number of terms** in the smallest sum whose value is ≥ `n`. The sum of 0 terms is equal to 0.

Hint: the first triangular number in the sequence (position 1) is 1. For `x > 1`, the triangular number at position `x` is equal to the triangular number at position `x-1` plus `x`.

Example: if the input is 25, then the output is 5 because 1 + 3 + 6 + 10 = 20 < 25 but 1 + 3 + 6 + 10 + 15 = 35 ≥ 25.

Your solution

```python
def my_function(n):
    # Write your solution here
    pass
```

Test your solution

```python
# Test case 1 (Example)
assert my_function(25) == 5

# Test case 2
assert my_function(56) == 6

# Test case 3
assert my_function(60) == 7

# Test case 4
assert my_function(1) == 1
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```python
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

Exercise 65: Classroom Attendance Tracker

Exercise Description

Imagine you're helping a school modernize its attendance system. Each classroom has a unique ID and a roster of students. The school wants to track attendance daily, marking students as present or absent.

Write a function `update_attendance(current_attendance, attendance_updates)` that:

- `current_attendance` is a dictionary where keys are classroom IDs and values are dictionaries mapping student names to their attendance status (`'Present'` or `'Absent'`).
- `attendance_updates` is a list of tuples `(class_id, student_name, status)` with the updated status.

The function should apply all updates and return the updated dictionary. If a classroom or student does not exist yet, add them.

Your solution

```
def update_attendance(current_attendance, attendance_updates):
    # Write your solution here
    pass
```

Test your solution

```python
# Test case 1 (Example)
current_attendance = {
    "Class 1A": {"Alice": "Present", "Bob": "Absent"},
    "Class 2B": {"Charlie": "Present", "Daisy": "Present"},
}

attendance_updates = [
    ("Class 1A", "Alice", "Absent"),
    ("Class 2B", "Eva", "Present"),
    ("Class 3C", "Frank", "Present"),
]

updated_attendance = update_attendance(current_attendance, attendance_updates)
assert updated_attendance == {
    "Class 1A": {"Alice": "Absent", "Bob": "Absent"},
    "Class 2B": {"Charlie": "Present", "Daisy": "Present", "Eva": "Present"},
    "Class 3C": {"Frank": "Present"},
}

# Test case 2
current_attendance = {
    "Class 3A": {"John": "Present", "Sarah": "Absent"},
    "Class 4B": {"Mike": "Absent", "Emily": "Present"},
}

attendance_updates = [
    ("Class 3A", "Sarah", "Present"),
    ("Class 4B", "Mike", "Present"),
    ("Class 5C", "Anna", "Present"),
]
```

```
updated_attendance = update_attendance(current_attendance, attendance_updates)
assert updated_attendance == {
    "Class 3A": {"John": "Present", "Sarah": "Present"},
    "Class 4B": {"Mike": "Present", "Emily": "Present"},
    "Class 5C": {"Anna": "Present"},
}
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

Exercise 66: Employee Skillset Tracker

Exercise Description

You are assisting the HR department of TechGiant Inc. to manage employees' skillsets.

Write a function `update_employee_skills(current_skills, skill_updates)` that:

- `current_skills` is a dictionary where keys are employee IDs and values are lists of skills.
- `skill_updates` is a list of tuples `(employee_id, list_of_skills, action)` where `action` is either `'add'` or `'remove'`.

The function should:

- If `action == 'add'`, **add** all skills in the list to that employee's skills (without removing existing ones).
- If `action == 'remove'`, **remove** each listed skill from the employee's skills (if present).
- If an employee ID is not present in `current_skills`, add it.
- If removing skills leaves an employee with no skills, remove that employee from the dictionary.

The function must return the updated dictionary.

Your solution

```
def update_employee_skills(current_skills, skill_updates):
    # Write your solution here
    pass
```

Test your solution

```
# Test case 1 (Example)
current_skills = {
    101: ["Python", "SQL"],
    102: ["Java", "C++", "Python"],
    103: ["HTML", "CSS"],
}

skill_updates = [
    (101, ["Java"], "add"),
    (104, ["Python", "JavaScript"], "add"),
    (102, ["C++"], "remove"),
    (103, ["HTML"], "remove"),
]

updated_skills = update_employee_skills(current_skills, skill_updates)
assert updated_skills == {
    101: ["Python", "SQL", "Java"],
    102: ["Java", "Python"],
    103: ["CSS"],
    104: ["Python", "JavaScript"],
}
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

Exercise 67: Inventory Management in Gridmart

Exercise Description

Gridmart is organized as a grid where each cell stores the quantity of a product category.

Write a function `update_inventory(current_inventory, updates)` that:

- `current_inventory` is a 2D list (list of lists) where each sublist is a row and each element is the quantity of a product category.
- `updates` is a list of tuples `(row, column, delta)` describing how many products to add (positive) or subtract (negative).

The function should:

- For each update, check if `(row, column)` is inside the grid. If any update refers to an invalid position, return `None`.
- Otherwise, apply all updates in order, modifying `current_inventory`.
- If after an update any quantity becomes negative, return `-1`.
- If all updates are valid and no quantity goes negative, return the updated 2D list.

Your solution

```python
def update_inventory(current_inventory, updates):
    # Write your solution here
    pass
```

Test your solution

```python
# Test case 1 (Example)
current_inventory = [[20, 15, 10], [30, 25, 5], [12, 8, 3]]
updates = [(0, 1, -5), (1, 0, 10), (2, 2, -2)]
assert update_inventory(current_inventory, updates) == [[20, 10, 10], [40, 25, 5], [1

# Test case 2: invalid column index
current_inventory = [[40, 30, 20], [25, 35, 45], [15, 10, 5]]
updates = [(0, 2, 5), (1, 3, -15), (2, 0, 10)]
assert update_inventory(current_inventory, updates) is None

# Test case 3: negative quantity after update
current_inventory = [[50, 60], [20, 30], [70, 80]]
updates = [(0, 0, -100), (1, 1, 20), (2, 0, -15)]
assert update_inventory(current_inventory, updates) == -1

# Test case 4
current_inventory = [[100, 200, 300], [400, 500, 600]]
updates = [(0, 1, -50), (1, 2, 25), (1, 0, -40)]
assert update_inventory(current_inventory, updates) == [[100, 150, 300], [360, 500, 6

# Test case 5
current_inventory = [[5, 10, 15], [20, 25, 30], [35, 40, 45], [50, 55, 60]]
updates = [(3, 2, -10), (2, 1, 20), (0, 0, 5)]
assert update_inventory(current_inventory, updates) == [[10, 10, 15], [20, 25, 30], [
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

Exercise 68: Library Book Tracker

Exercise Description

A library wants to track whether books are currently 'in' or 'out'.

Write a function `update_library(current_status, transactions)` that:

- `current_status` is a dictionary mapping book titles to their status (`'in'` or `'out'`).
- `transactions` is a list of tuples `(title, status)` where status is `'in'` or `'out'`.

The function should apply each transaction in order, updating or inserting entries in the dictionary, and then return the updated dictionary. If a title does not exist yet, add it.

Your solution

```python
def update_library(current_status, transactions):
    # Write your solution here
    pass
```

Test your solution

```python
# Test case 1 (Example)
current_status = {
    "The Great Gatsby": "in",
    "To Kill a Mockingbird": "out",
    "1984": "in",
}

transactions = [
    ("To Kill a Mockingbird", "in"),
    ("The Catcher in the Rye", "out"),
    ("1984", "out"),
]

updated_status = update_library(current_status, transactions)
assert updated_status == {
    "The Great Gatsby": "in",
    "To Kill a Mockingbird": "in",
    "1984": "out",
    "The Catcher in the Rye": "out",
}
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

Exercise 69: The Classroom Gradebook

Exercise Description

Mr. Alan maintains a gradebook where each student's test scores are stored in a row.

Write a function `calculate_student_averages(gradebook)` that:

- Receives a 2D list `gradebook`, where each inner list contains the scores of one student.
- Returns a list of tuples `(student_index, average_score)` where `student_index` is the index of the student in `gradebook` and `average_score` is their average rounded to 2 decimal places.

Caveats:

- Students with no scores are represented by an empty inner list: their average is defined to be 0.

Your solution

```
def calculate_student_averages(gradebook):
    # Write your solution here
    pass
```

Test your solution

```
# Test case 1 (Example)
gradebook = [[78, 82, 85], [92, 88, 91], [83, 76, 79], [95, 92]]
assert calculate_student_averages(gradebook) == [(0, 81.67), (1, 90.33), (2, 79.33),

# Further tests follow the patterns in the import notebook
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

Exercise 70: The Seating Arrangement Puzzle

Exercise Description

The community hall seating is organized as a 2D grid of 'Occupied' / 'Available' seats.

Write a function `update_seating_arrangement(current_seating, updates)` that:

- `current_seating` is a 2D list where each inner list is a row of seats.
- `updates` is a list of tuples `(row, col, new_status)` where `new_status` is `'Occupied'` or `'Available'`.

The function should:

- First check that the grid is regular (all rows have the same length). If not, return `-1`.
- For each update, if `(row, col)` is inside the grid, update that seat's status. If `(row, col)` is outside the grid, ignore that update.
- Return the updated 2D list.

Your solution

```
def update_seating_arrangement(current_seating, updates):
    # Write your solution here
    pass
```

Test your solution

```
# Test case 1 (Example)
current_seating = [
    ["Available", "Occupied", "Available"],
    ["Occupied", "Occupied", "Available"],
```

```
    ["Available", "Available", "Available"],
]

updates = [(0, 0, "Occupied"), (1, 2, "Occupied"), (2, 1, "Occupied")]
assert update_seating_arrangement(current_seating, updates) == [
    ["Occupied", "Occupied", "Available"],
    ["Occupied", "Occupied", "Occupied"],
    ["Available", "Occupied", "Available"],
]

# Additional tests mirror the import notebook behaviour
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

Exercise 71: Count Pairs

Exercise Description

Write a function `my_function(n, a, b, k)` that returns the **number of pairs** `(x, y)` such that:

1. `x` is a multiple of 3, strictly larger than 0 and smaller than or equal to `n`.
2. `y` is a number strictly larger than `a` and strictly smaller than `b`.
3. The difference between `x` and `y` is at most `k` (included), i.e. `x - y <= k`.

Your solution

```
def my_function(n, a, b, k):
    # Write your solution here
    pass
```

Test your solution

```
# Test case 1 (Example)
assert my_function(32, 9, 21, 12) == 95

# Test case 2
assert my_function(32, 3, 21, 12) == 130

# Test case 3
assert my_function(32, 3, 21, 3) == 79

# Test case 4
assert my_function(2, 3, 21, 3) == 0
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code**

before running the cell containing the Python Tutor link.

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

Exercise 72: Count Pairs Between

Exercise Description

Write a function `my_function(n, a, b, k)` that returns the **number of pairs** `(x, y)` such that:

1. `y` is an even number strictly larger than 0 and smaller than or equal to `n`.
2. `x` is a number strictly larger than `a` and strictly smaller than `b`.
3. The product `x * y` is at most `k` (included).

Your solution

```
def my_function(n, a, b, k):
    # Write your solution here
    pass
```

Test your solution

```
# Test case 1 (Example)
assert my_function(2, 1, 6, 10) == 4
```

```
# Test case 2
assert my_function(2, 10, 60, 100) == 40

# Test case 3
assert my_function(6, 2, 6, 100) == 9

# Test case 4
assert my_function(6, 2, 6, 1) == 0
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

Exercise 73: Maxsum Pairs

Exercise Description

Write a function `my_function(n, a, b, k)` that returns the **maximum sum** `x + y` taken among all pairs `(x, y)` such that:

- `y` is a multiple of 3, strictly larger than 0 and strictly smaller than `n`.
- `x` is a number strictly larger than `a` and strictly smaller than `b`.
- The difference between `x` and `y` is at least `k` (included), i.e. `x - y >= k`.

If no pair satisfies the conditions, the function should return 0.

Your solution

```
def my_function(n, a, b, k):
    # Write your solution here
    pass
```

Test your solution

```
# Test case 1 (Example)
assert my_function(20, 1, 21, 12) == 26

# Test case 2
assert my_function(3, 1, 21, 12) == 0

# Test case 3
assert my_function(20, 1, 210, 12) == 227

# Test case 4
assert my_function(20, 1, 6, 2) == 8
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

Exercise 74: Maxsum pairs Odd

Exercise Description

Write a function `my_function(n, a, b, k)` that returns the **maximum sum** `x + y` taken among all pairs `(x, y)` such that:

1. `x` is a positive odd number smaller than or equal to `n`.
2. `y` is a number strictly larger than `a` and smaller than or equal to `b`.
3. The product `x * y` is at most `k` (included).

If no pair satisfies the conditions, the function should return 0.

Your solution

```
def my_function(n, a, b, k):
    # Write your solution here
    pass
```

Test your solution

```
# Test case 1 (Example)
assert my_function(20, 1, 6, 2) == 3

# Test case 2
assert my_function(2, 1, 6, 2) == 3

# Test case 3
assert my_function(2, 1, 6, 20) == 7

# Test case 4
assert my_function(2, 1, 6, 1) == 0
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

Exercise 75: Difference between main and anti-diagonal

Exercise Description

Let `M` be an `n × m` matrix of numbers organized as a list-of-lists with `n` lists where each inner list has length `m`. You can assume that all inner lists have the same size `m` and you can assume `M` to be non-empty.

Write a Python function called `my_function(M)` that, given a matrix `M`, returns the absolute value of the difference between the sum of its main diagonal and the sum of its anti-diagonal.

Keep in mind the following caveats and definitions:

- normally, main diagonal and anti-diagonal are defined for square matrices only (namely matrices whose number of rows is equal to the number of columns): if `n` is not equal to `m`, the function must return `-1`
- the main diagonal of a matrix is defined as the list of entries lying from the top left to the bottom right corner of the matrix
- the anti-diagonal of a matrix is defined as the list of entries lying from the top right to the bottom left corner of the matrix

Your solution

```
def my_function(M):
    # compute number of rows n
    # compute number of columns m using the length of the first row
    # if n is not equal to m, return -1 (matrix is not square)
    # create a variable sum_diag initialised to 0
    # for each index i from 0 to n-1
        # add the element M[i][i] on the main diagonal to sum_diag
    # create a variable sum_antidiag initialised to 0
```

```python
    # for each index i from 0 to n-1
        # add the element M[i][n-1-i] on the anti-diagonal to sum_antidiag
    # if sum_antidiag is greater than sum_diag, return sum_antidiag - sum_diag
    # else if sum_antidiag is less than sum_diag, return sum_diag - sum_antidiag
    # otherwise (they are equal), return 0
    pass
```

Test your solution

```python
# Test case 1 (Example)
M = [[1, 2, 3], [4, 5, 6], [7, 8, 9]]
assert my_function(M) == 0

# Additional tests
M = [[1, 2, 3], [4, 5, 6], [7, 8, 9], [10, 11, 12]]
assert my_function(M) == -1

M = [
    [29, 10, 12, 33, 83, 28, 32, 81, 91, 84],
    [98, 60, 84, 68, 60, 18, 65, 10, 51, 50],
    [20, 15, 33, 33, 64, 37, 82, 50, 90, 45],
    [80, 38, 9, 43, 69, 82, 78, 39, 46, 44],
    [38, 69, 68, 20, 62, 53, 18, 59, 12, 12],
    [52, 14, 48, 3, 72, 64, 74, 45, 28, 97],
    [6, 27, 4, 2, 82, 67, 79, 96, 50, 55],
    [23, 83, 46, 45, 17, 26, 39, 48, 9, 75],
    [7, 58, 84, 63, 98, 25, 50, 78, 35, 30],
    [55, 36, 22, 32, 45, 14, 14, 5, 75, 34],
]
assert my_function(M) == 62
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

Exercise 76: Replace odd columns with previous max

Exercise Description

Let `M` be an n x m matrix of numbers organized as a list-of-lists with n lists where each inner list has length m. You can assume that all inner lists have the same size m and you can assume `M` to have at least 2 columns.

Write a Python function called `my_function(M)` that, given a matrix `M`, returns a modified version of `M` where the values in the odd columns are replaced with the maximum element of the previous even column.

Furthermore:

- if `n` is not equal to `m`, the function must return `-1`
- if `n` is equal to `m`, but there is at least one negative element (regardless of its position), the function must return `-2`

Your solution

```python
def my_function(M):
    # compute number of rows n
    # compute number of columns m using the length of the first row
    # if n is not equal to m, return -1 (matrix is not square)
    # for each row index i from 0 to n-1
        # for each column index j from 0 to m-1
            # if M[i][j] is negative, return -2
    # create a variable maxval initialised to 0
    # for each column index j from 0 to m-1
        # if j is even (including column 0)
            # create an empty list l
            # for each row index i from 0 to n-1
                # append M[i][j] to l
            # set maxval to the maximum value in l
        # otherwise (j is odd)
            # for each row index i from 0 to n-1
                # replace M[i][j] with maxval
    # return the modified matrix M
    pass
```

Test your solution

```python
# Test case 1 (Example)
M = [[1,2,3], [4,5,6], [7,8,9]]
assert my_function(M) == [[1, 7, 3], [4, 7, 6], [7, 7, 9]]
```

Test your solution

```python
# Additional tests
M = [[1, 2, 3], [4, 5, 6], [7, 8, 9], [10, 11, 12]]
assert my_function(M) == -1

M = [[1, 2, 3], [-0.01, 5, 6], [7, 8, 9]]
assert my_function(M) == -2

M = [
    [1, 2, 3, 0, 4],
    [3, 3, 1, 0, 3],
    [7, 3, 8, 4, 0],
    [1, 6, 7, 9, 0],
    [2, 4, 6, 9, 0],
]
assert my_function(M) == [
    [1, 7, 3, 8, 4],
    [3, 7, 1, 8, 3],
    [7, 7, 8, 8, 0],
    [1, 7, 7, 8, 0],
    [2, 7, 6, 8, 0],
]
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

Exercise 77: Replace column i with row i

Exercise Description

Write a function `my_function(M, i)` that is given a list of lists of numbers `M` and an integer number `i` such that:

- `M` represents a square matrix containing `D x D` elements: you can assume that all its sublists (corresponding to rows) have the same length and that the number `D` of rows equals the number `D` of columns.
- Integer `i` is a row and column index: it is larger than or equal to `0`, and strictly smaller than `D`.

The function should modify `M` by replacing the `i`-th column with the `i`-th row and should then return the modified matrix.

Back-of-the-envelope example: if `M = [[1,2,3],[4,5,6],[7,8,9]]` and `i = 1`, the returned matrix should be `[[1,4,3],[4,5,6],[7,6,9]]`.

Your solution

```python
def my_function(M, i):
    # Write your solution here
    pass
```

Test your solution

```python
# Test case 1 (Example)
a = my_function([[1, 2, 3], [4, 5, 6], [7, 8, 9]], 1)
assert a == [[1, 4, 3], [4, 5, 6], [7, 6, 9]]
```

Test your solution

```python
# Additional tests
a = my_function([[1, 2, 3], [4, 5, 6], [7, 8, 9]], 2)
assert a == [[1, 2, 7], [4, 5, 8], [7, 8, 9]]

a = my_function([[1, 2, 3, 4], [5, 6, 7, 8], [9, 10, 11, 12], [13, 14, 15, 16]], 0)
assert a == [[1, 2, 3, 4], [2, 6, 7, 8], [3, 10, 11, 12], [4, 14, 15, 16]]

a = my_function([[1, 2], [3, 4]], 1)
assert a == [[1, 3], [3, 4]]
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

Exercise 78: Replace row i with column j

Exercise Description

Write a function `my_function(M, i, j)` that is given a list of lists `M` of numbers and two integer numbers `i` and `j` such that:

- `M` represents a square matrix containing `D x D` elements: you can assume that all its sublists (corresponding to rows) have the same length and that the number `D` of rows equals the number `D` of columns.
- Integer `i` is a row index: it is larger than or equal to `0`, and strictly smaller than `D`.
- Integer `j` is a column index: it is larger than or equal to `0`, and strictly smaller than `D`.

The function should modify `M` by replacing the `i`-th row with the `j`-th column and should then return the modified matrix.

Back-of-the-envelope example: if `M = [[1,2,3],[4,5,6],[7,8,9]]` and `i = 1`, `j = 2`, the returned matrix should be `[[1,2,3],[3,6,9],[7,8,9]]`.

Your solution

```
def my_function(M, i, j):
    # Write your solution here
    pass
```

Test your solution

```
# Test case 1 (Example)
a = my_function([[1, 2, 3], [4, 5, 6], [7, 8, 9]], 1, 2)
assert a == [[1, 2, 3], [3, 6, 9], [7, 8, 9]]
```

Test your solution

```
# Additional tests
a = my_function([[1, 2, 3], [4, 5, 6], [7, 8, 9]], 2, 2)
assert a == [[1, 2, 3], [4, 5, 6], [3, 6, 9]]

a = my_function([[1, 2], [4, 5]], 0, 1)
assert a == [[2, 5], [4, 5]]

a = my_function([[1, 2, 3, 4], [5, 6, 7, 8], [9, 10, 11, 12], [13, 14, 15, 16]], 3, 
assert a == [[1, 2, 3, 4], [5, 6, 7, 8], [9, 10, 11, 12], [1, 5, 9, 13]]
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

Exercise 79: Replace row i with column i

Exercise Description

Write a function `my_function(M, i)` that is given a list `M` of lists of numbers and an integer number `i` such that:

- `M` represents a square matrix containing `D x D` elements: you can assume that all its sublists (corresponding to rows) have the same length and that the number `D` of rows equals the number `D` of columns.
- Integer `i` is a row and column index: it is larger than or equal to `0`, and strictly smaller than `D`.

The function should modify `M` by replacing the `i`-th row with the `i`-th column and should then return the modified matrix.

Back-of-the-envelope example: if `M = [[1,2,3],[4,5,6],[7,8,9]]` and `i = 2`, the returned modified matrix should be `[[1,2,3],[4,5,6],[3,6,9]]`.

Your solution

```python
def my_function(M, i):
    # Write your solution here
    pass
```

Test your solution

```python
# Test case 1 (Example)
a = my_function([[1, 2, 3], [4, 5, 6], [7, 8, 9]], 2)
assert a == [[1, 2, 3], [4, 5, 6], [3, 6, 9]]
```

Test your solution

```python
# Additional tests
a = my_function([[1, 2, 3, 4], [5, 6, 7, 8], [9, 10, 11, 12], [13, 14, 15, 16]], 3)
assert a == [[1, 2, 3, 4], [5, 6, 7, 8], [9, 10, 11, 12], [4, 8, 12, 16]]

a = my_function([[1, 2], [3, 4]], 0)
assert a == [[1, 3], [3, 4]]

a = my_function([[1, 2], [3, 4]], 1)
assert a == [[1, 2], [2, 4]]
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

Exercise 80: Sum of i-th row and i-th column

Exercise Description

Write a function `my_function(M, i)` that is given a list of lists of numbers `M` and an integer number `i` such that:

- `M` represents a square matrix containing `D x D` elements: you can assume that all of its sublists have the same length and that the number `D` of rows equals the number `D` of columns.
- Integer `i` is a row and column index: it is larger than or equal to `0`, and strictly smaller than `D`.

The function should return the sum of the elements in the `i`-th row and the elements in the `i`-th column. Element in row `i` and column `i` should be counted twice.

Back-of-the-envelope example: if `M = [[1, 2, 3], [4, 5, 8], [7, 8, 0]]` and `i = 2`, the code should return `26` because `7 + 8 + 0 + 3 + 8 + 0 = 26`.

Your solution

```python
def my_function(M, i):
    # Write your solution here
    pass
```

Test your solution

```python
# Test case 1 (Example)
a = my_function([[1, 2, 3], [4, 5, 8], [7, 8, 0]], 2)
assert a == 26
```

Test your solution

```python
# Additional tests
a = my_function([[1, 2, 3], [4, 5, 8], [1, 8, 0]], 0)
assert a == 12

a = my_function([[1, 2], [4, 5]], 1)
assert a == 16

a = my_function([[1, 2, 3, 0], [4, 5, 8, 0], [0, 8, 0, 0], [0, 8, 0, 0]], 0)
assert a == 11
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```python
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

Exercise 81: Sum of items in column range

Exercise Description

Write a function `my_function(L, i, j)` that is given a list `L` of lists of numbers and two integer numbers `i` and `j`. `L` represents a matrix with `R` rows and `C` columns (`R` and `C` are not given as input). You can assume that all the `R` sublists have the same length `C`, with `0 <= i < C` and `0 <= j < C`.

The function shall return the sum of items in columns between column `i` and column `j`, or `0` if the matrix is empty or `i > j`.

Back-of-the-envelope example: `my_function([[1,2,3,4],[10,20,30,40],[5,6,7,8]],1,2)` should return 68 (2+20+6+3+30+7).

Your solution

```python
def my_function(L, i, j):
    # Write your solution here
    pass
```

Test your solution

```
# Test case 1 (Example)
a = my_function([[1, 2, 3, 4], [10, 20, 30, 40], [5, 6, 7, 8]], 1, 2)
assert a == 68
```

Test your solution

```
# Additional tests
a = my_function([[1, 2, 3, 4], [10, 20, 30, 40], [5, 6, 7, 8]], 0, 1)
assert a == 44

a = my_function([[1, 2, 3, 4], [1, 2, 3, 4], [5, 6, 7, 8]], 0, 1)
assert a == 17

a = my_function([[1, 2, 3, 4], [1, 2, 3, 4], [5, 6, 7, 8], [10, 20, 30, 50]], 0, 2)
assert a == 90
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

Exercise 82: Sum of items in row range

Exercise Description

Write a function `my_function(L, i, j)` that is given a list of lists of numbers `L` and two integer numbers `i` and `j`. `L` represents a matrix with `R` rows and `C` columns (`R` and `C` are not given as input). You can assume that all the `R` sublists have the same length `C`, with `0 <= i < R` and `0 <= j < R`.

The function shall return the sum of items in rows between row `i` and row `j`, or `0` if the matrix is empty or `i > j`.

Back-of-the-envelope example: `my_function([[1,2,3,4],[10,20,30,40],[5,6,7,8]],1,2)` should return 126 (10+20+30+40+5+6+7+8).

Your solution

```
def my_function(L, i, j):
    # Write your solution here
    pass
```

Test your solution

```
# Test case 1 (Example)
a = my_function([[1, 2, 3, 4], [10, 20, 30, 40], [5, 6, 7, 8]], 1, 2)
assert a == 126
```

Test your solution

```python
# Additional tests
a = my_function([[1, 2, 3, 4], [10, 20, 30, 40], [5, 6, 7, 8]], 0, 1)
assert a == 110

a = my_function([[1, 2, 3, 4], [1, 2, 3, 4]], 0, 1)
assert a == 20

a = my_function([[1, 2, 3, 4], [1, 2, 3, 4], [5, 0, 0, 10]], 0, 2)
assert a == 35
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```python
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

Exercise 83: Sum of main diagonal

Exercise Description

Write a function `my_function(L)` that accepts a list of lists `L` of numbers. `L` represents a square matrix and you can assume that all sublists have the same length and that the number of rows equals the number of columns.

The function shall return the sum of items in the top-left to bottom-right diagonal, or `0` if the matrix is empty.

Back-of-the-envelope example: `my_function([[1,2,3],[1,2,3],[1,2,3]])` returns 6 (1+2+3).

Your solution

```
def my_function(L):
    # Write your solution here
    pass
```

Test your solution

```
# Test case 1 (Example)
assert my_function([[1, 2, 3], [1, 2, 3], [1, 2, 3]]) == 6
```

Test your solution

```
# Additional tests
assert my_function([]) == 0
assert my_function([[1, 1, 1], [1, 1, 1], [0, 0, -2]]) == 0

large_matrix = [
    [1, 2, 3, 4, 5, 6, 7, 8, 9, 10],
    [11, 12, 13, 14, 15, 16, 17, 18, 19, 20],
    [21, 22, 23, 24, 25, 26, 27, 28, 29, 30],
    [31, 32, 33, 34, 35, 36, 37, 38, 39, 40],
    [41, 42, 43, 44, 45, 46, 47, 48, 49, 50],
    [51, 52, 53, 54, 55, 56, 57, 58, 59, 60],
```

```
    [61, 62, 63, 64, 65, 66, 67, 68, 69, 70],
    [71, 72, 73, 74, 75, 76, 77, 78, 79, 80],
    [81, 82, 83, 84, 85, 86, 87, 88, 89, 90],
    [91, 92, 93, 94, 95, 96, 97, 98, 99, 100],
]
assert my_function(large_matrix) == 505
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

Exercise 84: Sum of top-right to bottom-left diagonal

Exercise Description

Write a function `my_function(L)` that accepts a list of lists `L` of numbers. `L` represents a square matrix and you can assume that all sublists have the same length and that the number of rows equals the number of columns.

The function shall return the sum of items in the top-right to bottom-left diagonal, or `0` if the matrix is empty.

Back-of-the-envelope example: `my_function([[1,2,3],[1,2,3],[1,2,3]])` returns 6 because 3 + 2 + 1 = 6.

Your solution

```
def my_function(L):
    # Write your solution here
    pass
```

Test your solution

```
# Test case 1 (Example)
a = my_function([[1, 2, 3], [1, 2, 3], [1, 2, 3]])
assert a == 6
```

Test your solution

```
# Additional tests
a = my_function([[10, 10, 3], [1, 10, 3], [10, 2, 3]])
assert a == 23

a = my_function([[10, 10, 3, 1], [1, 10, 3, 9], [10, 2, 3, 10], [0, 2, 3, 10]])
assert a == 6

a = my_function([])
assert a == 0
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

Exercise 85: Matrix transpose

Exercise Description

Let `M` be an `n × m` matrix of numbers organized as a list-of-lists with `n` lists where each inner list has length `m`.

Write a function called `my_function(M)` that takes as input a matrix `M` and returns its transpose.

Keep in mind the following caveats and definitions:

- if the input matrix is empty, the function must return `-1`
- the input list-of-lists must indeed be a valid matrix where all rows have the same length `m`: if this does not hold, the function must return `-2`
- the transpose of an `n × m` matrix is a new `m × n` matrix where the item in position `(i, j)` of the original matrix lies in position `(j, i)` in the transposed matrix

Your solution

```
def my_function(M):
    # if M is an empty list, return -1
    # compute the number of rows n
    # build a list m with the length of each row
    # if the set of values in m has size different from 1, return -2 (rows have diffe
    # otherwise, extract the common row length into m
    # create an empty list MT to store the transposed matrix
    # for each column index j from 0 to m-1
        # create an empty list row
        # for each row index i from 0 to n-1
            # append M[i][j] to row
        # append row to MT
    # return MT as the transpose of M
    pass
```

Test your solution

```
# Test case 1 (Example)
M = [[1, 2, 3], [4, 5, 6], [7, 8, 9], [10, 11, 12]]
assert my_function(M) == [[1, 4, 7, 10], [2, 5, 8, 11], [3, 6, 9, 12]]

# Additional tests
M = [
    [29, 10, 12, 33, 83, 28, 32, 81, 91, 84],
    [98, 60, 84, 68, 60, 18, 65, 10, 51, 50],
    [20, 15, 33, 33, 64, 37, 82, 50, 90, 45, 8],
    [80, 38, 9, 43, 69, 82, 78, 39, 46, 44],
    [38, 69, 68, 20, 62, 53, 18, 59, 12, 12],
```

```
    [52, 14, 48, 3, 72, 64, 74, 45, 28, 97],
    [6, 27, 4, 2, 82, 67, 79, 96, 55],
    [23, 83, 46, 45, 17, 26, 39, 48, 9, 75],
    [7, 58, 84, 63, 98, 25, 50, 78, 35, 30],
    [55, 36, 22, 32, 45, 14, 14, 5, 75, 34],
]
assert my_function(M) == -2

M = [
    [29, 10, 12, 33, 83, 28, 32, 81, 91, 84],
    [98, 60, 84, 68, 60, 18, 65, 10, 51, 50],
    [20, 15, 33, 33, 64, 37, 82, 50, 90, 45],
    [80, 38, 9, 43, 69, 82, 78, 39, 46, 44],
    [38, 69, 68, 20, 62, 53, 18, 59, 12, 12],
    [52, 14, 48, 3, 72, 64, 74, 45, 28, 97],
    [6, 27, 4, 2, 82, 67, 79, 96, 50, 55],
    [23, 83, 46, 45, 17, 26, 39, 48, 9, 75],
    [7, 58, 84, 63, 98, 25, 50, 78, 35, 30],
    [55, 36, 22, 32, 45, 14, 14, 5, 75, 34],
]
assert my_function(M) == [
    [29, 98, 20, 80, 38, 52, 6, 23, 7, 55],
    [10, 60, 15, 38, 69, 14, 27, 83, 58, 36],
    [12, 84, 33, 9, 68, 48, 4, 46, 84, 22],
    [33, 68, 33, 43, 20, 3, 2, 45, 63, 32],
    [83, 60, 64, 69, 62, 72, 82, 17, 98, 45],
    [28, 18, 37, 82, 53, 64, 67, 26, 25, 14],
    [32, 65, 82, 78, 18, 74, 79, 39, 50, 14],
    [81, 10, 50, 39, 59, 45, 96, 48, 78, 5],
    [91, 51, 90, 46, 12, 28, 50, 9, 35, 75],
    [84, 50, 45, 44, 12, 97, 55, 75, 30, 34],
```

```
]

M = []
assert my_function(M) == -1
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

Exercise 86: Max upper diagonal minus min lower diagonal

Exercise Description

Let `M` be an `n × m` matrix of numbers organized as a list-of-lists with `n` lists where each inner list has length `m`. You can assume that all inner lists have the same size `m` and you can assume `M` to be non-empty.

Write a Python function called `my_function(M)` that, given a matrix `M`, returns the difference between the maximum value inside the upper diagonal and the minimum value of the lower diagonal:

- the upper diagonal is defined as the diagonal of the matrix directly above the main diagonal
- the lower diagonal is defined as the diagonal of the matrix directly below the main diagonal
- if `n` is not equal to `m`, the function must return `-1`
- if `n` is equal to `m` but there is at least one negative element (regardless of its position), the function must return `-2`

Your solution

```
def my_function(M):
    # Write your solution here
    pass
```

Test your solution

```
# Test case 1 (Example)
M = [[1, 2, 3], [4, 5, 6], [7, 8, 9]]
assert my_function(M) == 2
```

Test your solution

```
# Additional tests
M = [[1, 2, 3], [4, 5, 6], [7, 8, 9], [10, 11, 12]]
assert my_function(M) == -1
```

```
M = [[1, 2, 3], [-0.01, 5, 6], [7, 8, 9]]
assert my_function(M) == -2

M = [
    [1, 2, 3, 0, 4],
    [3, 3, 1, 0, 3],
    [7, 3, 8, 4, 0],
    [1, 6, 7, 9, 0],
    [2, 4, 6, 9, 0],
]
assert my_function(M) == 1
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

Exercise 87: Max upper triangular minus min lower triangular

Exercise Description

Let `M` be an `n × m` matrix of numbers organized as a list-of-lists with `n` lists where each inner list has length `m`. You can assume that all inner lists have the same size `m` and you can assume `M` to be non-empty.

Write a Python function called `my_function(M)` that, given a matrix `M`, returns the difference between the maximum value of its upper triangular matrix and the minimum value of its lower triangular matrix.

Keep in mind the following caveats and definitions:

- the upper triangular matrix is defined as the portion of the matrix composed by only elements lying above (and including) the main diagonal
- the lower triangular is defined as the portion of the matrix composed by only elements lying below (and excluding in this case) the main diagonal
- normally, triangular matrices are special cases of square matrices (namely matrices whose number of rows is equal to the number of columns): if `n` is not equal to `m`, the function must return `-1`
- if the matrix is a square matrix but there is at least one negative element (regardless of its position), the function must return `-2`

Your solution

```
def my_function(M):
    # Write your solution here
    pass
```

Test your solution

```python
# Test case 1 (Example)
M = [[1,2,3], [4,5,6], [7,8,9]]
assert my_function(M) == 5
```

Test your solution

```python
# Additional tests
M = [[1, 2, 3], [4, 5, 6], [7, 8, 9], [10, 11, 12]]
assert my_function(M) == -1

M = [[1, 2, 3], [-0.01, 5, 6], [7, 8, 9]]
assert my_function(M) == -2

M = [
    [1, 2, 3, 0, 4],
    [3, 3, 1, 0, 3],
    [7, 3, 8, 4, 0],
    [1, 6, 7, 9, 0],
    [2, 4, 6, 9, 0],
]
assert my_function(M) == 8
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

Exercise 88: Auction

Exercise Description

Define a class Auction to manage bidding on items. The class shall provide: Constructor: takes item_name (string) and minimum_bid (float) as parameters. place_bid(bidder, amount): adds bid if higher than current highest. Returns True if bid accepted. withdraw_bid(bidder): removes bidder's bid. Returns True if bid existed. get_winner(): returns name of highest bidder (or "No winner" if no bids).

Your solution

```
# Write your solution here
```

Test your solution

```
# Test case 1 (Example)
a = Auction("Painting", 100.0)
a.place_bid("John", 150.0)
a.place_bid("Alice", 200.0)
a.place_bid("Bob", 175.0)
winner = a.get_winner()
print(winner)
```

```python
# Test case 2
a = Auction("Vase", 500.0)
a.place_bid("John", 400.0)
a.place_bid("Alice", 450.0)
winner = a.get_winner()
print(winner)

# Test case 3
a = Auction("Car", 1000.0)
a.place_bid("John", 1500.0)
a.place_bid("Alice", 2000.0)
a.withdraw_bid("Alice")
winner = a.get_winner()
print(winner)

# Test case 4
a = Auction("Watch", 200.0)
winner = a.get_winner()
print(winner)

# Test case 5
a = Auction("Phone", 300.0)
a.place_bid("John", 350.0)
a.place_bid("John", 400.0)
a.place_bid("Alice", 375.0)
winner = a.get_winner()
print(winner)
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

Exercise 89: Bank Account

Exercise Description

Create a Python class called `BankAccount` that represents a bank account. The class should have the following attributes and methods:

Attributes:

- `account_number` (string): The account number associated with the bank account.
- `balance` (float): The current balance of the bank account.

Methods:

- `deposit(amount)`: Accepts a float amount and adds it to the account's balance.
- `withdraw(amount)`: Accepts a float amount and subtracts it from the account's balance, if the balance is sufficient.
- `get_balance()`: Returns the current balance of the account as a float.
- `get_account_number()`: Returns the account number as a string.

Your solution

```
# Write your solution here
```

Test your solution

```
# Test case 1 (Example)
account = BankAccount("12345678", 100.0)
account.deposit(50.0)
print(account.get_balance())

# Test case 2
account = BankAccount("12345678", 100.0)
account.deposit(50.0)
account.withdraw(20.0)
print(account.get_balance())

# Test case 3
account = BankAccount("12345678", 100.0)
print(account.get_account_number())

# Test case 4
```

```
account = BankAccount("12345678", 100.0)
account.withdraw(200.0)
print(account.get_balance())
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

Exercise 90: Barrel

Exercise Description

Define a class `Barrel` to represent a barrel that contains a liquid. The barrel is initially full of liquid.

The class shall provide the following methods:

- Constructor: takes an input float `capacity`, that represents the total capacity of the barrel. Initially, the barrel contains an amount of liquid equal to its capacity.
- `draw(bottle_capacity)`: when called, draw liquid from the barrell to fill (as much as possible) a bottle with capacity `bottle_capacity`. The method shall return the amount of liquid actually drawn from the barrel.

Your solution

```
# Write your solution here
```

Test your solution

```
# Test case 1 (Example)
# We define a barrel containing 5 liters
b = Barrel(5)
# We fill a bottle containing 3 liters
b.draw(3)
# We try to fill another bottle containing 3 liters,
# but only 2 liters are left in the barrel
b.draw(3)
# We try to fill yet another bottle,
# but now barrel is empty
print(b.draw(3))
# Prints: 0

# Test case 2
a = Barrel(10)
b = a.draw(1)
```

```
print(b)

# Test case 3
a = Barrel(10)
b = a.draw(1)
b = a.draw(4)
print(b)

# Test case 4
a = Barrel(10)
b = a.draw(1)
b = a.draw(12)
print(b)
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

Exercise 91: Car

Exercise Description

Define a class `Car` to simulate the behaviour of a car. The user will be able to refuel the car, and then to drive, consuming fuel (if there is enough fuel).

The class shall provide the following methods:

- Constructor: takes one float parameter `fuel_consumption`, expressed in litres per kilometre. The fuel tank is initially empty.
- `refuel(quantity)`: when called, `quantity` litres of fuels are stored in the tank.
- `drive(distance)`: ask to drive for `distance` kilometres, while consuming fuel. The method shall return the number of kilometres actually travelled: if there is not enough fuel, the car travels as much as possible, stopping as soon as tank becomes empty.

Your solution

```
# Write your solution here
```

Test your solution

```
# Test case 1 (Example)
c = Car(0.1)    # Our car consumes 0.1 liters per kilometer
# Let's refuel 20 liters
c.refuel(20)
# Try to drive for 150 km
traveled = c.drive(150)
print(int(traveled))        # Expected: 150 - ok

# Test case 2
```

```
c = Car(0.1)
c.refuel(20)
traveled = c.drive(150)
traveled = c.drive(100)
print(float(traveled))

# Test case 3
c = Car(0.1)
c.refuel(20)
traveled = c.drive(150)
traveled = c.drive(100)
traveled = c.drive(70)
print(float(traveled))

# Test case 4
c = Car(0.1)
c.refuel(20)
traveled = c.drive(150)
traveled = c.drive(100)
traveled = c.drive(70)
c.refuel(30)
traveled = c.drive(70)
print(int(traveled))
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

Exercise 92: Cat

Exercise Description

Define a class `Cat` to represent cat characters in a videogame. A cat, as you know, has 9 lives.

The class shall provide the following methods:

- Constructor: does not take any parameters
- `kill()`: when called, the cat loses one of its lives
- `is_alive()`: returns `True`, if the cat is still alive (i.e., if it has lost at most 8 lives), `False` otherwise.

Your solution

```
# Write your solution here
```

Test your solution

```
# Test case 1 (Example)
c = Cat()
```

```python
c.kill()
c.kill()
c.kill()
c.kill()
c.kill()
c.kill()
c.kill()
c.kill()
c.kill()
c.kill()
print(c.is_alive())

# Test case 2
c = Cat()
c.kill()
c.kill()
c.kill()
c.kill()
c.kill()
c.kill()
print(c.is_alive())

# Test case 3
c = Cat()
print(c.is_alive())

# Test case 4
c = Cat()
c.kill()
c.kill()
c.kill()
```

```
c.kill()
c.kill()
c.kill()
c.kill()
c.kill()
c.kill()
print(c.is_alive())
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

Exercise 93: Counter

Exercise Description

Define a class `Counter` to track frequencies.

The class shall provide:

- `Constructor`: takes no parameters.
- `increment(item)`: adds 1 to the count for given `item`
- `decrement(item)`: subtracts 1 from count for given `item` (count can't go below 0)
- `get_most_frequent()`: returns the item with highest count (if tie, return `"tie"`)

Your solution

```
# Write your solution here
```

Test your solution

```
# Test case 1 (Example)
c = Counter()
c.increment("a")
c.increment("b")
c.increment("a")
c.increment("a")
most_freq = c.get_most_frequent()
print(most_freq)

# Test case 2
c = Counter()
c.increment("x")
c.increment("x")
c.decrement("x")
c.decrement("x")
c.decrement("x")
c.increment("y")
most_freq = c.get_most_frequent()
```

```python
print(most_freq)

# Test case 3
c = Counter()
c.increment("dog")
c.increment("cat")
c.increment("dog")
c.increment("cat")
most_freq = c.get_most_frequent()
print(most_freq)

# Test case 4
c = Counter()
c.increment("python")
c.increment("python")
c.decrement("python")
most_freq = c.get_most_frequent()
print(most_freq)

# Test case 5
c = Counter()
c.increment("red")
c.increment("blue")
c.increment("red")
c.decrement("blue")
c.increment("green")
most_freq = c.get_most_frequent()
print(most_freq)
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

Exercise 94: Inventory

Exercise Description

Define a class `Inventory` to manage product stock.

The class shall provide:

- Constructor: takes no parameters
- `add_product(product_id, quantity, price)`: adds new product with given `quantity`, unit `price` and `product_id`
- `sell(product_id, quantity)`: reduces `quantity` for the product `product_id` if available. Returns total price if successful, -1 if insufficient stock
- `get_most_valuable()`: returns `product_id` of product with highest (quantity × price).

Your solution

```
# Write your solution here
```

Test your solution

```
# Test case 1 (Example)
i = Inventory()
i.add_product("A123", 5, 10.0)
i.add_product("B456", 3, 20.0)
i.add_product("C789", 10, 5.0)
most_valuable = i.get_most_valuable()
print(most_valuable)

# Test case 2
i = Inventory()
i.add_product("laptop", 10, 1000.0)
i.add_product("mouse", 20, 25.0)
sale_price = "{:.2f}".format(i.sell("laptop", 2))
print(sale_price)

# Test case 3
i = Inventory()
i.add_product("phone", 5, 500.0)
i.sell("phone", 3)
result = i.sell("phone", 3)
print(result)

# Test case 4
i = Inventory()
i.add_product("X001", 10, 50.0)
i.add_product("X002", 5, 200.0)
i.sell("X001", 8)
most_valuable = i.get_most_valuable()
print(most_valuable)
```

```
# Test case 5
i = Inventory()
i.add_product("table", 3, 100.0)
i.add_product("chair", 8, 50.0)
sale_price = "{:.2f}".format(i.sell("chair", 2))
i.add_product("desk", 4, 150.0)
print(sale_price)
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

Exercise 95: Match

Exercise Description

Define a class `Match` to represent a soccer match.

The class shall provide the following methods:

- Constructor: takes two string parameters (home_team, away_team) with the name of playing teams
- `goal(team)`: called when a team scores a goal. String `team` shall be the name of the team which scored the goal
- `get_score()`: returns the current score, as a tuple with 4 elements `(home_team, home_score, away_team, away_score)`
- `get_winning()`: if a team is winning, returns the name of that team. Otherwise, returns `"Tie"`.

Your solution

```
# Write your solution here
```

Test your solution

```
# Test case 1 (Example)
m = Match("Liverpool", "Manchester")
m.goal("Manchester")
m.goal("Manchester")
m.goal("Liverpool")
w = m.get_winning()
print(w)

# Test case 2
m = Match("Liverpool", "Manchester")
m.goal("Manchester")
m.goal("Manchester")
```

```python
m.goal("Liverpool")
s = m.get_score()
print(s[1])

# Test case 3
m = Match("Liverpool", "Manchester")
m.goal("Manchester")
m.goal("Manchester")
m.goal("Liverpool")
m.goal("Liverpool")
print(m.get_winning())

# Test case 4
m = Match("Liverpool", "Manchester")
m.goal("Manchester")
m.goal("Manchester")
m.goal("Manchester")
m.goal("Manchester")
m.goal("Manchester")
m.goal("Manchester")
s = m.get_score()
print(s[3])
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pytl
```

Exercise 96: Padlock

Exercise Description

Define a class `Padlock`, where each instance represents a combination padlock. A padlock has two states: unlocked and locked. A locked padlock can be unlocked only if you know the code.

The class shall provide the following methods:

- Constructor: takes a string parameter code, which is the numeric code that will unlock the padlock. Initially the padlock is unlocked
- `is_locked()`: shall return `True` if the padlock is currently locked, `False` otherwise
- `lock()`: shall lock the padlock
- `unlock(code)`: called to try to unlock the padlock. If `code` is correct, the padlock shall become unlocked. If `code` is incorrect, the padlock state shall not change.

Your solution

```
# Write your solution here
```

Test your solution

```
# Test case 1 (Example)
p = Padlock('1234')
print(p.is_locked())

# Test case 2
p = Padlock('1234')
p.lock()
print(p.is_locked())

# Test case 3
p = Padlock('1234')
p.lock()
p.unlock('0000')
print(p.is_locked())

# Test case 4
p = Padlock('1234')
p.lock()
p.unlock('0000')
p.unlock('1234')
p.unlock('9999')
print(p.is_locked())
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

Exercise 97: Parking Lot

Exercise Description

Define a class `ParkingLot` to manage parking spaces.

The class shall provide:

- Constructor: takes number_of_spaces (int) as parameter
- `park_car(license_plate)`: assigns a space if available to a car identified by `license_plate`. Returns `True` if successful
- `remove_car(license_plate)`: removes car identified by `license_plate` from lot. Returns `True` if car was present
- `get_usage_percent()`: returns percentage of occupied spaces (0-100).

Your solution

```
# Write your solution here
```

Test your solution

```
# Test case 1 (Example)
p = ParkingLot(3)
p.park_car("ABC123")
p.park_car("DEF456")
usage = "{:.2f}".format(p.get_usage_percent())
print(usage)

# Test case 2
p = ParkingLot(2)
p.park_car("CAR111")
p.park_car("CAR222")
result = p.park_car("CAR333")
print(result)

# Test case 3
p = ParkingLot(4)
p.park_car("X1")
p.park_car("X2")
p.remove_car("X1")
p.park_car("X3")
usage = "{:.2f}".format(p.get_usage_percent())
print(usage)

# Test case 4
p = ParkingLot(5)
p.park_car("AA11")
result = p.remove_car("BB22")
print(result)

# Test case 5
p = ParkingLot(10)
```

```python
p.park_car("TEST1")
p.park_car("TEST2")
p.remove_car("TEST1")
p.remove_car("TEST2")
usage = "{:.2f}".format(p.get_usage_percent())
print(usage)
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```python
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

Exercise 98: Personal Wallet

Exercise Description

Define a class `Wallet` to represent a personal digital wallet. The user will be able to load money in the wallet, and then to spend money to purchase items.

The class shall provide the following methods:

- Constructor: does not take any parameter: it initialises an empty wallet (i.e., starting credit is 0)
- `load(amount)`: when called, the `amount` of money amount shall be added to the wallet credit
- `get_credit()`: returns the current balance of the wallet
- `purchase(price)`: purchases an item whose price is `price`. Returns `True` on success, i.e., if there is enough credit in the wallet the transaction is completed. Returns `False` if there is not enough credit, and the transaction is aborted (i.e., item is not purchased).

Your solution

```
# Write your solution here
```

Test your solution

```
# Test case 1 (Example)
w = Wallet()
# Let's load some money
w.load(10.00)
# We buy an item
success = w.purchase(6.00)
print(success)

# Test case 2
w = Wallet()
w.load(10.00)
success = w.purchase(6.00)
```

```
print(float(w.get_credit()))

# Test case 3
w = Wallet()
w.load(10.00)
success = w.purchase(6.00)
success = w.purchase(7.00)
print(success)

# Test case 4
w = Wallet()
w.load(10.00)
success = w.purchase(6.00)
success = w.purchase(7.00)
w.load(5.00)
success = w.purchase(7.00)
print(float(w.get_credit()))
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

Exercise 99: Scale

Exercise Description

Define a class `Scale` to represent a traditional scale (balance) with two plates: a left and a right plate. The user will be able to incrementally load each plate, adding weight, and test whether the scale is in equilibrium or it tilts to the left or to the right.

The class shall provide the following methods:

- Constructor: does not take any parameter: it initialises a scale with empty plates
- `load(weight, plate)`: when called, the weight `weight` shall be added to plate `plate`. `plate` is a string that may be `'L'` (left plate) or `'R'` (right plate)
- `measure()`: returns the current measurement outcome. The method returns a string that may be `'E'` (equilibrium), `'L'` (left plate is heavier) or `'R'` (right plate is heavier).

Your solution

```
# Write your solution here
```

Test your solution

```
# Test case 1 (Example)
s = Scale()
m = s.measure()
print(m)

# Test case 2
s = Scale()
```

```
m = s.measure()
s.load(2.5, 'L')
s.load(4, 'R')
m = s.measure()
print(m)

# Test case 3
s = Scale()
m = s.measure()
s.load(2.5, 'L')
s.load(4, 'R')
m = s.measure()
s.load(3, 'L')
m = s.measure()
print(m)

# Test case 4
s = Scale()
m = s.measure()
s.load(2.5, 'L')
s.load(4, 'R')
m = s.measure()
s.load(3, 'L')
m = s.measure()
s.load(1.5, 'R')
m = s.measure()
print(m)
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

Exercise 100: Student

Exercise Description

Define a class `Student` to track student grades.

The class shall provide:

- Constructor: takes name (string) as parameter
- `add_grade(grade)`: adds a new `grade` (float) to student's grades
- `get_average()`: returns the average of all grades
- `is_passing()`: returns `True` if average is >= 60, `False` otherwise.

Your solution

```
# Write your solution here
```

Test your solution

```
# Test case 1 (Example)
s = Student("Alice")
s.add_grade(45.5)
s.add_grade(55.5)
result = s.is_passing()
print(result)

# Test case 2
s = Student("John")
s.add_grade(85.5)
s.add_grade(92.0)
s.add_grade(88.5)
average = "{:.2f}".format(s.get_average())
print(average)

# Test case 3
s = Student("Bob")
s.add_grade(58.0)
s.add_grade(62.0)
average = "{:.2f}".format(s.get_average())
print(average)

# Test case 4
s = Student("Carol")
s.add_grade(75.5)
average = "{:.2f}".format(s.get_average())
print(average)

# Test case 5
s = Student("David")
s.add_grade(82.25)
```

```
s.add_grade(89.75)
s.add_grade(95.50)
s.add_grade(91.25)
average = "{:.2f}".format(s.get_average())
print(average)
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

Exercise 101: Telephone Account

Exercise Description

Define a class `TelephoneAccount` to keep track of the phone calls of a subscriber of a telephone line.

The class shall provide the following methods:

- Constructor: takes two float parameters (`call_setup_fee`, `per_minute_fee`)
- `add_call(duration)`: called when a phone call is performed. If `duration` is 0, the call is unsuccessful and nothing is charged. Otherwise, `duration` is the length of the call, ir minutes. The subscriber is charged the call setup fee, plus a variable cost, depending on the call duration
- `get_bill()`: returns the total price to be paid for all the calls that have been performed so far.

Your solution

```
# Write your solution here
```

Test your solution

```
# Test case 1 (Example)
a = TelephoneAccount(0.1, 0.2)
a.add_call(2)
a.add_call(0)
a.add_call(1.5)
b = a.get_bill()
print(float(b))

# Test case 2
a = TelephoneAccount(0.1, 0.2)
a.add_call(2)
a.add_call(0)
a.add_call(1)
b = a.get_bill()
```

```
a.add_call(1)
print(float(a.get_bill()))

# Test case 3
a = TelephoneAccount(1, 3)
a.add_call(2)
a.add_call(0)
a.add_call(1)
b = a.get_bill()
a.add_call(1)
print(float(a.get_bill()))

# Test case 4
a = TelephoneAccount(1, 3)
a.add_call(2)
a.add_call(0)
a.add_call(0)
b = a.get_bill()
a.add_call(1)
print(float(a.get_bill()))
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

Exercise 102: Temperature

Exercise Description

Define a class `Temperature` to handle temperature conversions.

The class shall provide:

- Constructor: takes temperature value (float) and unit ('C' or 'F' as string)
- `to_celsius()`: returns temperature converted to Celsius if in Fahrenheit, or same value if already Celsius
- `to_fahrenheit()`: returns temperature converted to Fahrenheit if in Celsius, or same value if already Fahrenheit
- `is_freezing()`: returns `True` if temperature is at or below freezing point (0°C/32°F), `False` otherwise.

Please note that: C = (5/9) * (F - 32) and F = (C * 9/5) + 32

Your solution

```
# Write your solution here
```

Test your solution

```
# Test case 1 (Example)
t = Temperature(0.0, 'C')
```

```python
result = t.is_freezing()
print(result)

# Test case 2
t = Temperature(98.6, 'F')
celsius = "{:.2f}".format(t.to_celsius())
print(celsius)

# Test case 3
t = Temperature(25.0, 'C')
fahrenheit = "{:.2f}".format(t.to_fahrenheit())
print(fahrenheit)

# Test case 4
t = Temperature(20.0, 'F')
celsius = "{:.2f}".format(t.to_celsius())
print(celsius)

# Test case 5
t = Temperature(100.0, 'F')
fahrenheit = "{:.2f}".format(t.to_fahrenheit())
print(fahrenheit)
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

Exercise 103: Text Message

Exercise Description

Define a class `TextMessage` to handle message character limits.

The class shall provide:

- Constructor: takes character_limit (int) as parameter
- `write_message(text)`: sets message `text` if within limit. Returns `True` if successful, `False` if text exceeds limit
- `add_prefix(prefix)`: adds `prefix` to start of message if total length stays within limit. Returns `True` if successful and `False` otherwise
- `get_length()`: returns current message length.

Your solution

```
# Write your solution here
```

Test your solution

```python
# Test case 1 (Example)
t = TextMessage(10)
t.write_message("Hello")
length = t.get_length()
print(length)

# Test case 2
t = TextMessage(5)
result = t.write_message("Too long")
print(result)

# Test case 3
t = TextMessage(10)
t.write_message("Test")
result = t.add_prefix("My ")
print(result)

# Test case 4
t = TextMessage(6)
t.write_message("World")
result = t.add_prefix("Hello ")
print(result)

# Test case 5
t = TextMessage(20)
length = t.get_length()
print(length)
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

Exercise 104: Thermostat

Exercise Description

Define a class `Thermostat` to control room temperature.

The class shall provide:

- Constructor: takes `current_temp` and `target_temp` (both float) as parameters
- `adjust(amount)`: adjusts the current temperature by a given `amount` (float). Returns the updated current temperature
- `set_target(temp)`: sets new target temperature `temp`
- `get_status()`: returns `"Heating"` if current < target, `"Cooling"` if current > target, `"Stable"` if equal.

Your solution

```
# Write your solution here
```

Test your solution

```
# Test case 1 (Example)
t = Thermostat(25.5, 23.0)
t.set_target(22.0)
status = t.get_status()
print(status)

# Test case 2
t = Thermostat(18.50, 21.00)
t.adjust(1.00)
status = t.get_status()
print(status)

# Test case 3
t = Thermostat(19.5, 21.0)
t.adjust(1.5)
current = "{:.2f}".format(t.adjust(0.0))
print(current)

# Test case 4
t = Thermostat(23.00, 23.00)
t.adjust(0.00)
status = t.get_status()
print(status)

# Test case 5
t = Thermostat(22.5, 23.0)
t.adjust(-1.0)
t.adjust(0.75)
```

```
current = "{:.2f}".format(t.adjust(0.5))
print(current)
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

Exercise 105: Vote

Exercise Description

Define a class `Vote` to manage a simple voting system.

The class shall provide:

- Constructor: takes list of candidates (strings) as parameter
- `cast_vote(candidate)`: adds one vote for given `candidate` if valid. Returns `True` if valid candidate, `False` otherwise
- `get_votes(candidate)`: returns the number of votes for given `candidate`
- `get_winner()`: returns candidate with most votes (if tie, return `"tie"`).

Your solution

```
# Write your solution here
```

Test your solution

```
# Test case 1 (Example)
v = Vote(["Alice", "Bob", "Charlie"])
v.cast_vote("Alice")
v.cast_vote("Bob")
v.cast_vote("Alice")
winner = v.get_winner()
print(winner)

# Test case 2
v = Vote(["Red", "Blue"])
result = v.cast_vote("Green")
print(result)

# Test case 3
v = Vote(["X", "Y", "Z"])
v.cast_vote("X")
v.cast_vote("Y")
v.cast_vote("Y")
votes = v.get_votes("Y")
print(votes)

# Test case 4
v = Vote(["Left", "Right"])
v.cast_vote("Left")
```

```
v.cast_vote("Right")
winner = v.get_winner()
print(winner)

# Test case 5
v = Vote(["A", "B", "C"])
winner = v.get_winner()
print(winner)
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

Exercise 106: Average scores with validation

Exercise Description

Write a function `average_scores(scores)` that receives a list of numbers and returns their average.

The function should:

- raise a `ValueError` if the list is empty
- raise a `TypeError` if any element is not an `int` or `float`
- otherwise, compute and return the average using a loop (do not use the built-in `sum`)

Your solution

```
def average_scores(scores):
    # Write your solution here
    pass
```

Test your solution

```
# Tests for valid lists
assert average_scores([10, 20, 30]) == 20
assert average_scores([5.0, 7.0]) == 6.0
```

Test your solution

```
# Tests for invalid inputs (expect exceptions)
try:
    average_scores([])
    assert False, 'Expected ValueError for empty list'
except ValueError:
    print('OK: ValueError raised for empty list')

try:
    average_scores([10, 'bad', 20])
    assert False, 'Expected TypeError for non-numeric score'
```

```
except TypeError:
    print('OK: TypeError raised for non-numeric score')
```

```
OK: ValueError raised for empty list
OK: TypeError raised for non-numeric score
```

```
The Kernel crashed while executing code in the current cell or a previous c
ell.

Please review the code in the cell(s) to identify a possible cause of the f
ailure.

Click <a href='https://aka.ms/vscodeJupyterKernelCrash'>here</a> for more i
nfo.

View Jupyter <a href='command:jupyter.viewOutput'>log</a> for further detai
ls.
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

Exercise 107: BankAccount class with input validation (raise exceptions)

Exercise Description

Define a class `BankAccount` that represents a simple bank account.

The class should:

- be initialized with an `owner` (string) and an optional `balance` (number, default 0)
- raise a `TypeError` if `owner` is not a string or `balance` is not a number
- raise a `ValueError` if `balance` is negative
- provide a method `deposit(amount)` that:
    - raises a `TypeError` if `amount` is not a number
    - raises a `ValueError` if `amount` is less than or equal to 0
    - otherwise, adds `amount` to the balance
- provide a method `withdraw(amount)` that:
    - raises a `TypeError` if `amount` is not a number
    - raises a `ValueError` if `amount` is less than or equal to 0
    - raises a `ValueError` if `amount` is greater than the current balance
    - otherwise, subtracts `amount` from the balance

This exercise focuses on **raising** exceptions from methods when inputs are invalid. Do not catch exceptions inside the class.

Your solution

```
class BankAccount:
    def __init__(self, owner, balance=0):
        # Write your solution here
        pass

    def deposit(self, amount):
        # Write your solution here
        pass

    def withdraw(self, amount):
        # Write your solution here
        pass
```

Test your solution

```
# Basic tests for valid operations
account = BankAccount('Alice', 100)
account.deposit(50)
assert account.balance == 150
account.withdraw(20)
assert account.balance == 130
```

Test your solution

```python
# Tests for invalid operations (expect exceptions)
try:
    BankAccount(123, 0)
    assert False, 'Expected TypeError for non-string owner'
except TypeError:
    print('OK: TypeError raised for non-string owner')

try:
    BankAccount('Bob', -10)
    assert False, 'Expected ValueError for negative balance'
except ValueError:
    print('OK: ValueError raised for negative balance')

account = BankAccount('Carol', 50)
try:
    account.deposit(0)
    assert False, 'Expected ValueError for non-positive deposit'
except ValueError:
    print('OK: ValueError raised for non-positive deposit')

try:
    account.withdraw(100)
    assert False, 'Expected ValueError for withdrawing too much'
except ValueError:
    print('OK: ValueError raised for withdrawing too much')
```

```
OK: TypeError raised for non-string owner
OK: ValueError raised for negative balance
OK: ValueError raised for non-positive deposit
OK: ValueError raised for withdrawing too much
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

Exercise 108: Group grades by band with error handling

Exercise Description

Write a function `group_grades_by_band(grades)` that receives a dictionary mapping student names (strings) to exam grades (numbers from 0 to 30) and returns a new dictionary that groups names by grade band.

Use the following bands (keys of the result dictionary):

- `'fail'` for grades 0..17
- `'pass'` for grades 18..23
- `'excellent'` for grades 24..30

The function should:

- raise a `TypeError` if `grades` is not a dictionary
- raise a `TypeError` if any key is not a string or any value is not an `int` or `float`
- raise a `ValueError` if any grade is outside the range 0..30
- otherwise, build and return a dictionary like `{'fail': [...], 'pass': [...], 'excellent': [...]}` using loops and dictionary operations

You may assume that each student appears at most once in the input dictionary.

Your solution

```python
def group_grades_by_band(grades):
    # Write your solution here
    pass
```

```
The Kernel crashed while executing code in the current cell or a previous c
ell.

Please review the code in the cell(s) to identify a possible cause of the f
ailure.

Click <a href='https://aka.ms/vscodeJupyterKernelCrash'>here</a> for more i
nfo.

View Jupyter <a href='command:jupyter.viewOutput'>log</a> for further detai
ls.
```

Test your solution

```python
# Tests for valid grades
grades = {'Alice': 16, 'Bob': 20, 'Carol': 28}
grouped = group_grades_by_band(grades)
assert 'Alice' in grouped['fail']
assert 'Bob' in grouped['pass']
assert 'Carol' in grouped['excellent']
assert len(grouped['fail']) == 1
assert len(grouped['pass']) == 1
assert len(grouped['excellent']) == 1
```

Test your solution

```python
# Tests for invalid input (expect exceptions)
try:
    group_grades_by_band([])
    assert False, 'Expected TypeError for non-dict grades'
except TypeError:
    print('OK: TypeError raised for non-dict grades')

try:
    group_grades_by_band({123: 10})
    assert False, 'Expected TypeError for non-string name'
except TypeError:
    print('OK: TypeError raised for non-string name')

try:
    group_grades_by_band({'Alice': 'bad'})
    assert False, 'Expected TypeError for non-numeric grade'
except TypeError:
    print('OK: TypeError raised for non-numeric grade')
```

```python
try:
    group_grades_by_band({'Alice': 40})
    assert False, 'Expected ValueError for grade > 30'
except ValueError:
    print('OK: ValueError raised for grade > 30')
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```python
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

Exercise 109: Normalize student scores with dictionaries

Exercise Description

Write a function `normalize_scores(scores)` that receives a dictionary mapping student names (strings) to exam scores (numbers from 0 to 30). It should return a new dictionary mapping each name to a normalized score between 0.0 and 1.0 (inclusive).

The function should:

- raise a `TypeError` if `scores` is not a dictionary
- raise a `ValueError` if the dictionary is empty
- raise a `TypeError` if any key is not a string or any value is not an `int` or `float`
- raise a `ValueError` if any score is outside the range 0..30
- otherwise, return a new dictionary where each score is divided by 30.0

Use loops and dictionary operations; do not use dictionary comprehensions.

Your solution

```
def normalize_scores(scores):
    # Write your solution here
    pass
```

Test your solution

```
# Tests for valid dictionaries
scores = {'Alice': 30, 'Bob': 15}
normalized = normalize_scores(scores)
assert normalized['Alice'] == 1.0
assert normalized['Bob'] == 0.5
assert len(normalized) == 2
```

Test your solution

```
# Tests for invalid dictionaries (expect exceptions)
try:
    normalize_scores([])
```

```python
    assert False, 'Expected TypeError for non-dict scores'
except TypeError:
    print('OK: TypeError raised for non-dict scores')

try:
    normalize_scores({})
    assert False, 'Expected ValueError for empty dict'
except ValueError:
    print('OK: ValueError raised for empty dict')

try:
    normalize_scores({123: 10})
    assert False, 'Expected TypeError for non-string name'
except TypeError:
    print('OK: TypeError raised for non-string name')

try:
    normalize_scores({'Alice': 'bad'})
    assert False, 'Expected TypeError for non-numeric score'
except TypeError:
    print('OK: TypeError raised for non-numeric score')

try:
    normalize_scores({'Alice': 40})
    assert False, 'Expected ValueError for score > 30'
except ValueError:
    print('OK: ValueError raised for score > 30')
```

```
OK: TypeError raised for non-dict scores
OK: ValueError raised for empty dict
OK: TypeError raised for non-string name
```

```
OK: TypeError raised for non-numeric score
OK: ValueError raised for score > 30
```

```
The Kernel crashed while executing code in the current cell or a previous c
ell.

Please review the code in the cell(s) to identify a possible cause of the f
ailure.

Click <a href='https://aka.ms/vscodeJupyterKernelCrash'>here</a> for more i
nfo.

View Jupyter <a href='command:jupyter.viewOutput'>log</a> for further detai
ls.
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

Exercise 110: Order service that catches and raises exceptions

Define two classes: `Inventory` and `OrderService`.

The `Inventory` class should:

- be initialized with a dictionary `stock` mapping product names (strings) to quantities (integers)
- raise a `TypeError` if the initial stock is not a dictionary, or if any key is not a string or any value is not an integer
- raise a `ValueError` if any quantity is negative
- provide a method `reserve(product, quantity)` that:
    - raises a `TypeError` if `product` is not a string or `quantity` is not an integer
    - raises a `ValueError` if `quantity` is less than or equal to `0`
    - raises a `KeyError` if the product is not in stock
    - raises a `ValueError` if there is not enough stock available
    - otherwise, decreases the stock by `quantity`

The `OrderService` class should:

- be initialized with an `Inventory` instance
- provide a method `place_order(product, quantity)` that:
    - calls `inventory.reserve(product, quantity)` inside a try/except block
    - catches `KeyError` and raises a new `KeyError` with a more descriptive message (e.g., `'Unknown product: ...'`)
    - catches `ValueError` and raises a new `ValueError` with a message like `'Cannot place order: ...'`
    - lets other unexpected exceptions propagate without catching them

This exercise focuses on **catching exceptions from another class, then raising new exceptions with more context**.

Your solution

```
class Inventory:
    def __init__(self, stock):
        # Write your solution here
        pass

    def reserve(self, product, quantity):
        # Write your solution here
        pass

class OrderService:
    def __init__(self, inventory):
        # Write your solution here
        pass
```

```python
    def place_order(self, product, quantity):
        # Write your solution here
        pass
```

Test your solution

```python
# Tests for valid orders
inv = Inventory({'apple': 10, 'banana': 5})
service = OrderService(inv)
assert service.place_order('apple', 3) is True
assert inv.stock['apple'] == 7
```

Test your solution

```python
# Tests for error cases (catch, then raise new exceptions)
inv = Inventory({'apple': 2})
service = OrderService(inv)

try:
    service.place_order('orange', 1)
    assert False, 'Expected KeyError for unknown product'
except KeyError as e:
    print('OK: KeyError raised with message:', e)

try:
    service.place_order('apple', 5)
    assert False, 'Expected ValueError for insufficient stock'
except ValueError as e:
    print('OK: ValueError raised with message:', e)
```

```
OK: KeyError raised with message: 'Unknown product: orange'
OK: ValueError raised with message: Cannot place order: not enough stock
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

Exercise 111: Recursive factorial with input validation (easy)

Exercise Description

Write a recursive function `safe_factorial(n)` that computes the factorial of `n`.

The function should:

- raise a `TypeError` if `n` is not an integer
- raise a `ValueError` if `n` is negative
- return `1` if `n` is 0
- otherwise, use recursion: `n * safe_factorial(n - 1)`

Your solution

```
def safe_factorial(n):
    # Write your solution here
    pass
```

Test your solution

```
# Tests for valid inputs
assert safe_factorial(0) == 1
assert safe_factorial(1) == 1
assert safe_factorial(4) == 24
assert safe_factorial(5) == 120
```

Test your solution

```
# Tests for invalid inputs (expect exceptions)
try:
    safe_factorial(-1)
    assert False, 'Expected ValueError for negative n'
except ValueError:
    print('OK: ValueError raised for negative n')
```

```
try:
    safe_factorial(3.5)
    assert False, 'Expected TypeError for non-int n'
except TypeError:
    print('OK: TypeError raised for non-int n')
```

```
OK: ValueError raised for negative n
OK: TypeError raised for non-int n
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

Exercise 112: Recursive nested dictionary lookup (hard)

Exercise Description

Write a recursive function `safe_get_nested(mapping, keys)` that receives:

- `mapping`: a (possibly nested) dictionary
- `keys`: a list of keys representing a path inside the nested dictionary

It should return the value found by following the sequence of keys. For example:

- `safe_get_nested({'a': {'b': 3}}, ['a', 'b'])` should return `3`.

The function should:

- raise a `TypeError` if `mapping` is not a dictionary
- raise a `TypeError` if `keys` is not a list
- raise a `ValueError` if `keys` is empty
- raise a `KeyError` if any key in the path is missing
- use recursion: at each step, take the first key and call itself on the nested dictionary with the remaining keys
- not use loops (implement the traversal purely with recursion)

Your solution

```python
def safe_get_nested(mapping, keys):
    # Write your solution here
    pass
```

Test your solution

```python
# Tests for valid lookups
data = {'a': {'b': {'c': 5}}, 'x': 1}
assert safe_get_nested(data, ['a', 'b', 'c']) == 5
assert safe_get_nested(data, ['x']) == 1
```

Test your solution

```python
# Tests for invalid inputs (expect exceptions)
try:
    safe_get_nested('not a dict', ['a'])
    assert False, 'Expected TypeError for non-dict mapping'
except TypeError:
    print('OK: TypeError raised for non-dict mapping')

try:
    safe_get_nested({}, 'not a list')
    assert False, 'Expected TypeError for non-list keys'
except TypeError:
    print('OK: TypeError raised for non-list keys')

try:
    safe_get_nested({}, [])
    assert False, 'Expected ValueError for empty keys list'
except ValueError:
    print('OK: ValueError raised for empty keys list')

try:
    safe_get_nested({'a': {}}, ['a', 'missing'])
    assert False, 'Expected KeyError for missing nested key'
except KeyError:
    print('OK: KeyError raised for missing nested key')
```

```
OK: TypeError raised for non-dict mapping
OK: TypeError raised for non-list keys
```

```
OK: ValueError raised for empty keys list
OK: KeyError raised for missing nested key
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

Exercise 113: Recursive sum of nested list with validation (medium)

Exercise Description